Agentic AI Orchestration Definition of Terms

Artificial Intelligence and Its Close Relationship to Machine Learning and Data Science

Artificial intelligence (AI) is not just a marketing term for machine learning (ML) and data science, though they are closely related and often overlap. [1, 2]

Key Differences and Relationships

- **Artificial Intelligence (AI):** The broad, overarching goal of creating computer systems or machines that can mimic human-like intelligence, reasoning, decision-making, and problem-solving. [1, 2, 3]
- **Machine Learning (ML):** A specific subset of AI focused on algorithms that allow computers to learn patterns from data and improve over time without being explicitly programmed for every scenario. ML is the core engine or technology that makes many modern AI applications work. [1, 2, 3, 4]
- **Data Science:** A broader multidisciplinary field focused on extracting insights, running statistical analysis, and making predictions from structured or unstructured data. While data science uses machine learning as a tool, it also encompasses traditional statistics, data cleaning, and business intelligence reporting that do not involve AI. [1, 2, 3]

Why the Marketing Confusion Exists

- **Rebranding Technology:** Companies frequently use "AI" as a buzzword to market software, analytics, or machine learning models because it sounds more futuristic and impressive to consumers than "statistical algorithms" or "machine learning". [1, 2, 3]
- **Product vs. Engine:** AI is typically the user-facing product or experience (such as a chatbot, recommendation system, or voice assistant), whereas machine learning and data science are the technical methods used behind the scenes to build it. [1, 2, 3]

Origin of the term “Artificial Intelligence”

I was curious about her claim that AI started as a marketing term. After some searching, I got to the Wikipedia page on the 1956 Dartmouth Workshop, which says that:

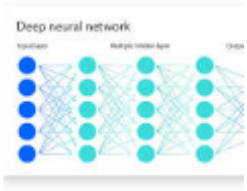
> *In 1955, John McCarthy, then a young Assistant Professor of Mathematics at Dartmouth College, decided to organize a group to clarify and develop ideas about thinking machines. He picked the name ‘Artificial Intelligence’ for the new field. He chose the name partly for its neutrality; avoiding a focus on narrow automata theory, and avoiding cybernetics which was heavily focused on*

analog feedback, as well as him potentially having to accept the assertive Norbert Wiener as guru or having to argue with him.

Neural Network

A neural network (or artificial neural network) is a machine learning model inspired by the structure and function of the human brain. It consists of interconnected units called artificial neurons (or nodes) that process data, recognize patterns, and make predictions. [1, 2, 3, 4]

Neural Networks Compared to Traditional Software



Traditional software (Software 1.0) runs on explicit, human-written rules and step-by-step logic, while neural networks (Software 2.0) learn patterns and parameters automatically from data. [1, 2, 3]

Core Differences

- **Programming Approach:**
  - **Traditional Software:** Programmers write explicit algorithms and logical statements (e.g., in C, Java, or Python) to transform inputs into outputs.
  - **Neural Networks:** The architecture defines a general structure, and training compiles a dataset into internal weights and biases to model the behavior. [1, 2, 3, 4]
- **Handling Data:**
  - **Traditional Software:** Excels at structured data, database lookups, mathematical calculations, and deterministic logic.
  - **Neural Networks:** Excels at unstructured, complex data like images, audio, and natural language by discovering features autonomously. [1, 2, 3]
- **Transparency & Interpretability:**
  - **Traditional Software:** High interpretability; every execution step can be traced, debugged, and audited.
  - **Neural Networks:** Low interpretability (often called a "black box"), making it difficult to pinpoint why a specific decision was made. [1, 2]

- **Execution & Performance:**
  - **Traditional Software:** Fast, predictable execution on standard CPUs with low resource requirements.
  - **Neural Networks:** Computationally expensive training and inference, frequently requiring specialized hardware like GPUs or TPUs. [1, 2, 3]
- **Flexibility & Maintenance:**
  - **Traditional Software:** Rigid; changing behavior requires rewriting or updating the source code.
  - **Neural Networks:** Adaptive; behavior can be updated or expanded by retraining on new or updated datasets. [1, 2, 3]

Summary Comparison Table

Feature	Traditional Software	Neural Networks
Logic Source	Human-coded rules	Data-driven training
Best Suited For	Deterministic calculations, logic, databases	Perception, pattern recognition, NLP
Explainability	High (Transparent)	Low (Black box)
Hardware	Standard CPUs	GPUs, TPUs, custom chips
Maintenance	Manual code refactoring	Dataset curation and model retraining

Next Token Prediction

Next-token prediction is an AI training and generation task where a model analyzes a sequence of text and calculates a probability distribution to estimate the single most likely token (a word, part of a word, number, or punctuation mark) that should come next. [1, 2, 3]

Token

In the context of agentic AI, a **token** is the fundamental unit of data—such as a word fragment, syllable, character, or punctuation mark—that an AI model reads, writes, and processes. [1, 2, 3]

Unlike a standard chatbot where a user enters a single prompt and gets a single response, **agentic AI** uses tokens recursively. An autonomous agent plans, calls tools, loops through trial-and-error, reflects, and self-corrects across multiple hidden steps. [1, 2]

Key Characteristics of Tokens in Agentic AI

- **Working Capital of Intelligence:** Tokens act as the operational currency and computing cost that power an agent's multi-step reasoning loops.
- **Recursive Multiplier:** Because agents generate internal monologues, plan sub-tasks, execute tool calls, and review errors before finishing a job, they consume significantly more tokens per task (often **5 to 30 times more**) than a standard linear prompt-response chat.
- **Context Window Pressure:** An agent's "context window" (its active memory limit measured in tokens) can fill up rapidly during a long, multi-step agentic workflow as history, tool outputs, and intermediate thoughts accumulate.
- **Token Inflation & Waste:** Unoptimized agentic workflows often suffer from token waste—such as redundant context re-sending or infinite looping—driving up latency and operational expenses.
- **Agent Tokenomics:** Enterprises manage token consumption as a specialized financial and operational metric to track the true cost of autonomous work. [1, 2, 3, 4, 5]

Embedding

An **embedding** in Agentic AI is a dense numerical vector (a list of numbers) that captures the semantic meaning of data—such as text, code, or user history—so that computers can measure how similar or related different pieces of information are. [1, 2, 3]

How It Works

- **Meaning into Math:** Unstructured data like words, sentences, or tool descriptions are run through a neural network model to convert them into coordinates in a high-dimensional vector space. [1, 2, 3]
- **Proximity Equals Similarity:** Concepts with similar meanings are mapped close together in this mathematical space (for example, "dog" and "puppy" sit much closer than "dog" and "tractor"). [1]

Role in Agentic AI Systems

In autonomous AI agents, embeddings move beyond simple text search to power core operational functions:

- **Retrieval from Knowledge Bases (RAG):** When an agent needs external facts, it embeds the user's query and searches a Vector Database to pull the most relevant documents or context chunks. [1, 2, 3]
- **Cross-Session Memory:** Agents use embeddings to store, index, and recall past user interactions, preferences, or task states over long timelines. [1]
- **Semantic Routing:** Instead of hardcoding if/then statements, an agent uses embeddings to compare a user's request against the descriptions of various tools, APIs, or sub-agents to dynamically pick the right one to handle the job. [1]
- **Personalization:** Agents compare real-time user actions against historical behavioral embeddings to tailor responses and decisions to a specific user's patterns. [1]

AI Model (Large Language Model)

A Large Language Model (LLM) is a type of artificial intelligence algorithm that uses deep learning and neural networks to understand, summarize, translate, and generate human-like text and code by predicting the next word or token in a sequence. [1, 2, 3]

Transformer Architecture

Built on deep learning transformer models that use "self-attention" mechanisms to process words in relation to every other word simultaneously rather than step-by-step. [1, 2]

Generative Pretrained Transform (GPT)

A **generative pre-trained transformer** (**GPT**) is a type of large language model (LLM)[1][2][3] that is widely used in generative artificial intelligence chatbots.[4][5] GPTs are based on a deep learning architecture called the transformer. They are pre-trained on large datasets of unlabeled content, and able to generate novel content.[2][3]
https://en.wikipedia.org/wiki/Generative_pre-trained_transformer

Pre-Training

The model acts as a general-purpose token predictor. It reads trillions of words from the internet and books to learn grammar, facts, and language patterns. It learns *how language works*, but it does not yet know how to be a helpful assistant.

Post Training - Why is this such a big deal that so many talk about?

Post-training is the second major phase of developing a Large Language Model (LLM), where a raw, pre-trained base model is adapted and refined to follow instructions, converse naturally, and behave safely. [1, 2, 3]

Conversational AI

Conversational AI is a set of technologies that allows computers to understand, process, and respond to human language in a natural, back-and-forth dialogue via text or voice. [1, 2, 3] Conversational AI combines several core technologies to manage a full interaction: [1]

Agentic AI

Agentic AI is an autonomous or semi-autonomous artificial intelligence system that can set its own goals, break them down into multi-step plans, and independently execute tasks using tools and APIs without needing constant human oversight. [1, 2, 3]

A major aspect of agentic AI is ‘took; use’.

AI Agent

An **AI agent** is an autonomous software system that uses a large language model (LLM) or artificial intelligence to reason about a goal, make a plan, and take multi-step actions using tools and APIs until the task is complete. [1, 2]

A common shorthand in the industry is: **Agent = Model + Harness**. While the model acts as the "brain" (handling reasoning and text generation), the harness layer acts as the "body" or operational scaffolding that lets the model interact with the real world. [1, 2, 3, 4, 5]

AI Subagent

An **AI subagent** is a specialized, subordinate AI agent that a primary or orchestrator agent spawns to handle a specific part of a larger task. [1, 2]

- **Hierarchical Delegation:** The main agent acts like a manager or project lead. It breaks a complex user goal into smaller chunks and delegates them to subagents. [1, 2, 3]
- **Context Isolation:** Each subagent typically operates in its own separate context window with tailored instructions, a specific set of tools, and limited permissions. It does not inherit the main conversation's full history. [1, 2]
- **Result Return:** The subagent completes its assigned job independently and sends only the final distilled output or summary back to the main orchestrator. [1, 2]

AI Agent/Subagent Initial Amnesia

Yes, a standard AI agent effectively experiences a form of amnesia upon initial activation or at the start of a fresh session, because the underlying Large Language Model (LLM) is inherently stateless. [1, 2]

Why AI Agents Start with Amnesia

- **Stateless Models:** An LLM does not retain memory of past runs or previous interactions on its own. Every time it generates a response, it resets or processes only what is explicitly handed to it in that specific prompt or context window. [1, 2]
- **Empty Working Memory:** When an agent boots up for the first time or launches a new session, its immediate context window is blank unless an external system injects data into it. [1, 2]
- **No Built-in Lived Experience:** Unlike humans, an AI agent does not possess continuous, background consciousness or natural biological memory; it does not "remember" what happened yesterday unless it reads from a database, file, or memory layer. [1, 2]

How Developers Fix Agent Amnesia

To prevent agents from starting completely blind, engineers build external memory and storage layers around them, categorized by frameworks like CoALA: [1]

- **System Prompts & Files:** Loading baseline instructions, rules, or files (such as `MEMORY.md` or configuration files) into the opening prompt. [1, 2]
- **Retrieval-Augmented Generation (RAG):** Connecting the agent to vector databases or knowledge graphs so it can pull up relevant facts or past logs upon startup. [1, 2]
- **Episodic & Semantic Memory Layers:** Using specialized middleware (such as Mem0 or MemGPT/Letta) to save and retrieve past interactions across multiple sessions. [1, 2]

If you are designing or deploying an AI agent, let me know:

- **What framework** or platform you are using (e.g., LangChain, CrewAI, Custom)?
- **What kind of information** you need the agent to remember between sessions?

Harness

An AI harness (or agent harness) is the software infrastructure and scaffolding that wraps around a stateless Large Language Model (LLM) to turn it into an active AI agent. [1, 2, 3]

Harness Layer

The "harness layer" refers to the software infrastructure and runtime environment that wraps around a foundation language model to turn it into an autonomous AI agent. [1, 2]

What the Harness Layer Does

- **Execution Loops:** Manages the dynamic back-and-forth loop of calling the model, interpreting its output, triggering tool calls, handling errors, and deciding when a task is complete. [1]
- **Tool & API Integration:** Connects the model to external systems, databases, code interpreters, or APIs (such as via the Model Context Protocol). [1, 2]
- **Memory & State Management:** Keeps track of conversation history, handles context window compaction, and manages persistent storage or checkpoints. [1, 2, 3]
- **Guardrails & Governance:** Enforces security boundaries, identity/credentials, rate limits, and requires human approval before executing high-stakes actions. [1, 2]
- **Observability & Logging:** Tracks reasoning traces, records telemetry, and audits what the agent actually did at runtime. [1, 2]

Harness vs. Framework

- **Frameworks** (like LangChain) provide modular building blocks, APIs, and primitives for developers to assemble their own workflows.
- **Harnesses** (like Claude Code or specialized runtimes) are opinionated, production-ready systems that ship with the execution loop already built-in and configured for reliable operation. [1, 2]

The shifting focus toward the **"harness layer"** marks a major maturation point in AI investing, as the performance gap between leading AI models narrows and raw tokens become heavily commoditized. As highlighted in research from Goldman Sachs, the harness layer acts as the vital infrastructure and "control plane" that houses, manages, and connects AI models—allowing them to access proprietary data, manage state, route queries efficiently, and securely follow complex enterprise workflows. [1, 2, 3]

Why the Value is Shifting to the Harness Layer

- **Model Convergence:** The technical performance differentiation between proprietary frontier models is shrinking.

- **Token Commoditization:** Industry leaders note that enterprise reliance is shifting toward open-source models, driving down the unit economic value of raw model outputs.
- **Enterprise ROI:** According to KPMG pulse data, **86% of mature organizations** seeing realized AI ROI utilize a formal AI harness layer to enforce data sovereignty and governance. [1, 2, 3]

Core Components of the Harness Layer

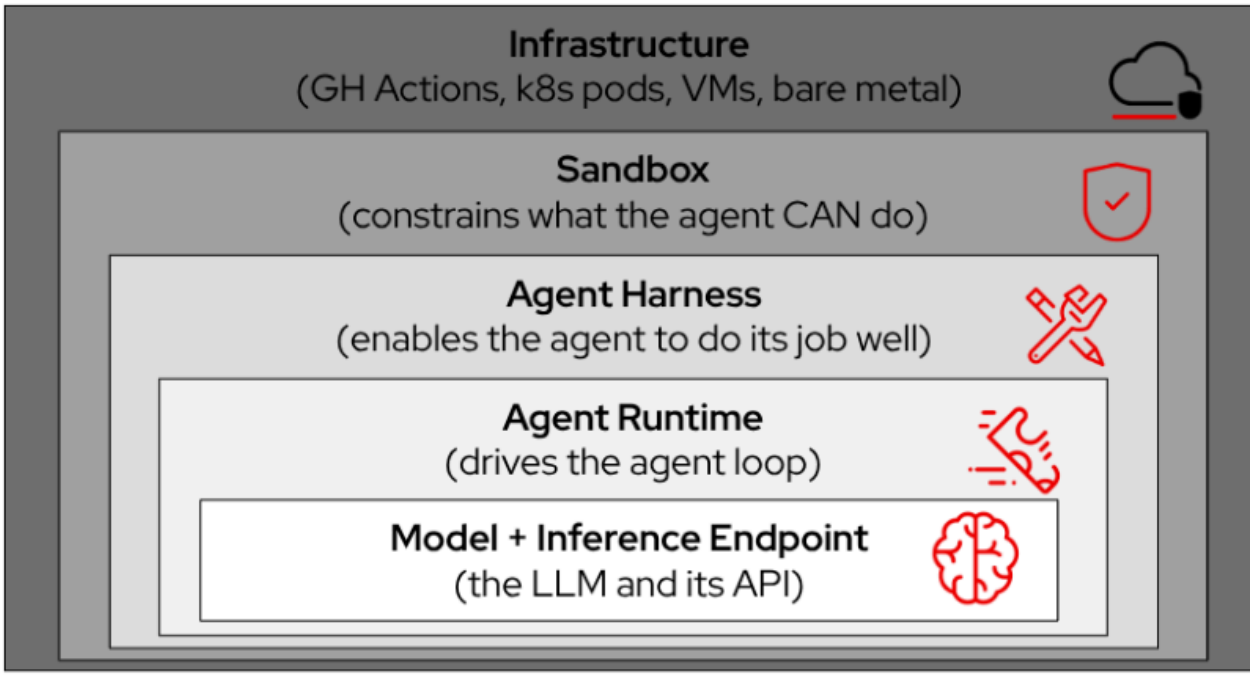
- **Orchestration & Workflow:** Functions like a horse harness, guiding the underlying model through multi-step corporate tool usage and memory retention.
- **Model Routers:** Intelligent meta-harness systems, such as Databricks' open-source Omnigen framework, that dynamically split tasks and route queries to the most cost-efficient or specialized model.
- **Verification & Security:** Guardrails that ensure compliance, monitor outputs for hallucinations, and handle enterprise access control. [1, 2, 3, 4]

Strategic Investment Implications

- **Beyond the Model Labs:** Traditional venture investment is diversifying past foundational model labs (like OpenAI or Anthropic) toward enablers building sovereign intelligence wrappers. [1]
- **The New Enterprise Lock-In:** While models are becoming interchangeable, the software harness driving the user experience and internal workflow integration is where true ecosystem stickiness is created. [1]

Builder Harness versus User Harness

The structural baseline for the concept comes from Birgitta Böckeler's April 2026 article, which elegantly defines an agent as `model + harness = agent`. She bifurcated the stack into a builder harness (the inner runtime shipped with the tool) and a user harness (the developer's custom context). This definition built on a wave of discussion from February 2026, which included Mitchell Hashimoto's pragmatic approach to engineering `AGENTS.md`contexts, OpenAI's overview of internal harness engineering for automated deployment, and Böckeler's original summary memo.



Infrastructure → sandbox → agent harness → runtime → model

Each layer has a different owner, a different failure mode, and a different design principle. Let's walk through them.

Source: https://www.redhat.com/en/blog/what-even-harness-ai

Workspace

In the context of agentic AI, a **workspace** (or agentic workspace) is a shared digital environment where human users and autonomous AI agents collaborate as peers on ongoing projects. [1, 2, 3]

Unlike a standard chat interface—where an AI only responds passively to isolated prompts and the conversation vanishes when closed—an agentic workspace acts as a persistent operating system for work. [1, 2]

Key Characteristics

- **Shared Surface:** Humans and AI agents work within the same context, viewing the same task boards, comment threads, documents, and codebases rather than chatting in a side window. [1, 2]
- **Persistent Memory & Artifacts:** Information, learned preferences, code, files, and outputs persist across sessions. An agent can generate a report or script and save it to a shared folder for another agent or human to pick up. [1, 2]

- **Delegated Execution & Proactivity:** Agents are not just conversational assistants; they have delegated authority to run code, use connected apps, search databases, schedule tasks, and push workflows forward autonomously. [1, 2, 3]
- **Permissions & Governance:** Because agents can trigger real-world actions or alter files, the workspace includes access controls, activity traces, and human-in-the-loop review guardrails. [1, 2, 3]

Examples in Practice

- **Team Project Spaces:** Platforms like Monday.com or Fast.io provide shared folders and task boards where research agents gather data, writing agents draft content, and humans review the final results. [1, 2]
- **Developer & Operations Workspaces:** Features like OpenAI ChatGPT Workspace Agents allow agents to access files, run code, and handle multi-step requests inside enterprise workflows. [1, 2]

Deterministic

In software engineering, **deterministic** describes a system where the same input will always produce the exact same output, following a predictable, fixed path. When comparing legacy (traditional) software to Large Language Models (LLMs), determinism is the defining line between rigid rules and probabilistic behavior.

Legacy Software vs. LLMs

Feature	Legacy Software (Deterministic)	LLMs (Probabilistic / Non-Deterministic)
Core Mechanism	Hard-coded logic, conditional statements (`if/else`), and explicit algorithms.	Statistical weights and mathematical probabilities trained on massive datasets.
Output Predictability	**100% predictable.** Inputting `2 + 2` will always yield `4` .	**Variable.** The same prompt can yield different phrasing, tone, or even facts across separate runs.
Execution Path	Follows a strict, traceable, step-by-step code path.	Navigates a complex neural network; paths are emergent and difficult to trace.
Handling of Ambiguity	Fails or throws an error if an input doesn't match expected formats.	Flexibly interprets vague, messy, or creative inputs to generate a plausible response.
Testing Approach	Unit tests with exact assertions (e.g., Assert `Output == Expected`).	Evaluation frameworks that check for semantic meaning, tone, and accuracy ranges.

Probabilistic

From the perspective of AI Large Language Models (LLMs), **probabilistic** means the model does not retrieve a single hard-coded "correct" answer. Instead, it calculates and assigns a mathematical probability distribution over a vocabulary of possible next tokens (words or sub-words) based on the context that came before it. [1, 2, 3]

Probabilistic Sampling: LLMs are built to be generative and creative rather than rigid. Instead of picking a single fixed answer, they output probabilities and use sampling parameters—such as `temperature` and `top_p`—to introduce controlled randomness. [1, 2]

Prompt

In the context of agentic AI, a **prompt** is an instruction, goal, or framework of rules given to an AI agent that defines how it thinks, plans, uses tools, and executes multi-step tasks. [1, 2, 3]

Unlike a simple chat prompt (which just asks a question or requests a single-turn answer), an agentic prompt acts as a blueprint for an autonomous loop. [1, 2, 3]

Key Components of a Prompt in Agentic AI

- **System Prompts / Base Instructions:** Hidden rules, personas, behavioral guardrails, and operational boundaries that stay active across the entire interaction. [1, 2]
- **Goal or Objective:** The high-level mission the agent is expected to achieve (e.g., *"investigate this bug and deploy a fix"*) rather than a static text output. [1]
- **Decision Framework & Tool Instructions:** Instructions telling the agent what external tools it has access to (like web searches, APIs, or code interpreters), when to use them, and how to interpret their results. [1]
- **Reasoning Architectures:** Prompts often structure how the agent thinks through a problem using methods like:
    - **Chain-of-Thought (CoT):** Forcing the agent to break down complex tasks into step-by-step logic.
    - **ReAct (Reasoning + Acting):** Interleaving internal thought processes with external actions and observations.
    - **Prompt Chaining:** Feeding the output of one prompt directly into the next stage of an automated workflow. [1, 2]

Prompt Engineering

In the context of agentic AI (systems where an AI model runs in a feedback loop, uses tools, maintains memory, and takes autonomous actions), prompt engineering is the practice of writing and structuring the operational instructions, tool definitions, reasoning loops, and guardrails that act as a job description and standard operating procedure for the agent**.** [1]

Unlike simple chatbot prompting—which focuses on asking a single clever question or managing a back-and-forth text conversation—agent prompt engineering governs how an autonomous agent behaves across multi-step execution. [1]

Core Elements of Prompt Engineering in Agentic AI

- **System Prompts and Role Definition:** Establishing the agent's core identity, boundaries, and objective. It is akin to onboarding a new hire and explaining their exact job responsibilities.

- **Tool Descriptions:** Clearly writing out the schemas, parameters, and descriptions for the external APIs, databases, or calculators the agent is allowed to invoke. If the tool description is vague, the agent will misuse it or hallucinate parameters.
- **Reason-Act (ReAct) Loops:** Guiding the model on *how* to think, plan, act, and observe results iteratively before jumping to a final conclusion (often structured via chain-of-thought prompting).
- **Memory and State Management Integration:** Instructing the agent on how to handle short-term conversational context, summarize past turns, or query long-term vector databases without losing its core mission.
- **Stop Conditions and Guardrails:** Defining explicit rules for when the agent must stop, ask for human confirmation, or abort a loop to prevent infinite execution or drifting off-task. [1, 2, 3]

Prompt Engineering vs. Context Engineering in Agents

In modern agentic workflows, prompt engineering overlaps closely with **context engineering**: [1, 2, 3]

- **Prompt Engineering** asks: *How should the model behave and interpret its instructions for this task?*
- **Context Engineering** asks: *What data, memory, tool outputs, and state are injected into the model's window at this exact moment?* [1, 2, 3, 4]

An agent's prompt often acts as a dynamic template that gets stitched together in real time with retrieved documents (via RAG), conversation history, and live tool responses. [1]

The Evolution: From Prompt Engineering to Context Engineering

In multi-turn agentic systems, static prompts are no longer enough. Practitioners often use **context engineering**—the practice of managing everything that enters the agent's working memory (including system prompts, past conversation history, tool outputs, and retrieved data) so the agent stays on track over long, complex workflows without losing its way or hallucinating. [1, 2, 3]

You can read more about how system design is evolving in guides like Effective context engineering for AI agents - Anthropic and Prompt Engineering for Agentic AI Systems. [1, 2]

Context Window

In agentic AI, the **context window** is the maximum amount of data (measured in tokens—roughly chunks of words or characters) that an AI model can "see," process, and reason about during a single inference pass or working-memory cycle. [1, 2, 3]

Think of it as the AI agent's active desk or short-term working memory. Anything that fits on the desk can be used; anything that doesn't fit is invisible to the model. [1, 2, 3, 4]

What Fills an Agent's Context Window?

Unlike a simple chat assistant, an AI agent stuffs multiple competing categories of information into this single finite space: [1]

- **System Instructions:** Role definitions, core behavioral rules, and safety constraints.
- **Agent Goal / Task:** The current objective or user command.
- **Tool Definitions & Schemas:** API specifications, available functions, and action parameters.
- **Conversation History:** Past turns or steps in the workflow.
- **Retrieved Knowledge (RAG):** Documents or database snippets pulled in dynamically.
- **Execution State / Scratchpad:** Intermediate thoughts, tool outputs, partial calculations, and working notes. [1, 2]

Why the Context Window Matters for Agents

- **Rapid Token Burn:** Agents make multiple back-and-forth tool calls and loops to complete a task. A multi-step workflow can burn through hundreds of thousands of tokens very quickly as context accumulates. [1]
- **Invisible Failures:** Once the context window fills up, older information is silently dropped or overwritten. An agent doesn't always raise an alarm; it may just keep running with missing pieces and give a confident yet wrong answer. [1]
- **Middle-Context Degradation:** Even with massive context windows (such as 1 million+ tokens), models often struggle to pay attention to details buried right in the middle of the text. [1, 2, 3]
- **Context Engineering:** Because stuffing everything into the window causes noise, high costs, and latency, developers use **context engineering**—compacting tool outputs, summarizing old chat turns, or offloading data to external vector stores/notes (agentic memory) rather than relying solely on a bigger window. [1, 2, 3, 4]

Loop Engineering

Loop engineering is the practice of designing automated, self-directed execution cycles for AI agents so they can continuously act, evaluate their own work, and re-prompt themselves until they reach a goal. [1, 2, 3]

Instead of a human manually typing a new prompt at every single step, loop engineering builds a system around the agent to handle the entire trial-and-error cycle on its own. [1, 2, 3, 4, 5]

How It Compares to Other AI Practices

- **Prompt Engineering:** Focuses on crafting the single best instruction for one-off AI interactions.
- **Harness Engineering:** Builds the technical environment and toolset wrapped around the model.
- **Loop Engineering:** Takes that harness and programs the repeating logic—telling the agent when to run, how to check its output, how to fix its mistakes, and when to stop. [1, 2, 3, 4, 5]

As explained in a breakdown by IBM, loop engineering replaces rigid prompt chains with dynamic, flexible loops built for long-running autonomous tasks. [1]

Core Building Blocks of a Loop

A well-designed loop usually includes these parts:

- **Clear Goals and Stop Conditions:** Rules that tell the agent what "done" looks like so it does not run forever or waste money on endless tokens.
- **Environment Tools and Connectors:** APIs or terminal access (such as Model Context Protocol tools) that let the agent actually execute code or fetch data.
- **Worktrees and Isolation:** Separate workspace directories (similar to Git worktrees) so multiple agent loops can run tasks at the same time without breaking each other's files.
- **Sub-agents and Checkers:** Splitting the workflow into separate roles—where one agent writes or acts, and a second agent reviews or tests the work.
- **Context Management:** Strategies to compact or refresh conversation history so the agent does not suffer from "context rot" over long jobs. [1, 2, 3, 4, 5, 6, 7]

Common Use Cases

- Autonomous software engineering (writing, testing, and debugging code)
- Automated bug fixing and responding to GitHub issues or support tickets
- Multi-step background research and data-processing pipelines [1, 2, 3, 4, 5]

Community Perspective

Opinions on the term vary among developers. Most users on Reddit agree that "loop engineering" is largely a new label for established software concepts like feedback loops,

automated retry logic, and cron-style scheduled jobs applied to modern large language models. [1]

Tool

In the context of agentic AI, a **tool** is a discrete function or piece of code (such as an API, database query, calculator, or file reader) that an AI agent can invoke to interact with the external world and perform actions. [1, 2, 3]

While a standard language model can only generate text based on its internal training, giving it tools allows it to execute real-world tasks. [1, 2]

How Tools Work

- **The Division of Labor:** The large language model (LLM) handles the **reasoning** (deciding *what* needs to be done and *which* tool to pick), while the application code runs the actual tool and returns the result.
- **Tool Calling (Function Calling):** The model outputs a structured request (like a JSON payload) specifying the tool name and required arguments. The agent framework catches this request, runs the function, and feeds the output back into the model's context.
- **The Agent Loop:** The process operates in a cycle—**plan, act, observe, and repeat**—until the model decides it has enough information or has completed the goal. [1, 2, 3, 4, 5]

Common Categories of Agentic Tools

- **Web Search:** Fetching live or up-to-date facts from the internet.
- **APIs & Integrations:** Sending emails, booking flights, or creating support tickets in external software.
- **Databases & Retrieval:** Querying internal company records or vector stores (Retrieval-Augmented Generation / GraphRAG).
- **Computation & Code Execution:** Running math calculations or executing code in a sandboxed interpreter.
- **File Operations:** Reading, writing, or editing local or cloud documents. [1, 2, 3, 4, 5, 6]

For more details on how models structure and execute these capabilities, see the Claude Tool Use Documentation or developer breakdowns like Understanding Tools in Agentic AI and Understanding Tools in the Agentic Framework. [1, 2, 3]

Skill

In modern AI agent frameworks (such as the open-source **Agent Skills** standard), an **AI skill** is a reusable, modular package that teaches an AI how to perform a specific task. [1, 2] **How they**

work: They are usually built around a `SKILL.md` file containing a name, a description, and step-by-step instructions (procedural knowledge). They can also bundle helper scripts (Python, JavaScript), reference files, or API connectors. [1, 2]

Ontology

An **ontology** in artificial intelligence is a formal, machine-readable model that defines the concepts, categories, properties, and relationships within a specific domain or subject area. [1, 2, 3]

Key Components of an AI Ontology

An ontology organizes knowledge through several core building blocks:

- **Classes (Concepts):** Groups or types of things in a domain (e.g., *Customer*, *Product*, *Hospital*).
- **Individuals (Instances):** Specific, real-world examples of a class (e.g., *John Doe* as an instance of *Customer*).
- **Properties (Attributes):** Characteristics, traits, or values that describe classes and individuals (e.g., a *Customer* has a *credit score*).
- **Relationships (Edges):** How different concepts connect to one another (e.g., a *Customer places* an *Order*).
- **Axioms (Rules):** Logical constraints and rules used to derive new facts or restrict invalid states (e.g., if a subscription is canceled, the customer status updates automatically). [1, 2, 3, 4, 5]

Why Ontologies Matter in AI

- **Eliminating Ambiguity:** Large language models (LLMs) are probabilistic and can guess or "hallucinate" meanings. An ontology acts as a "semantic contract" or "world model" that gives the AI hard definitions of what terms mean. [1, 2, 3]
- **Powering Knowledge Graphs:** An ontology provides the master schema or blueprint, while a Knowledge Graph populates that blueprint with actual operational data. [1, 2]
- **Enabling Reasoning:** Because ontologies contain explicit rules and relationships, an AI agent can logically infer conclusions that were never explicitly written out in a database table. [1, 2, 3]
- **Improving Search & RAG:** They anchor Retrieval-Augmented Generation (RAG) and semantic search engines so that looking up one term (like *heart attack*) automatically pulls in related medical equivalents (like *myocardial infarction*). [1, 2]

Would you like an example of how an ontology is built for a specific industry (like healthcare, finance, or e-commerce), or do you want to know how it differs from a simple database schema?

Frank Coyle is a computer science professor who teaches generative AI and Large Language Models (LLMs) at UC Berkeley and discusses the role of ontologies in agentic AI systems. [1, 2]

Key details of his work on the subject:

- **Core Argument:** He re-introduced ontologies as "logical guardrails" and "data as graphs" to help manage the probabilistic reasoning and hallucinations of LLMs. [1, 2]
- **Neurosymbolic AI:** He highlights combining the probabilistic power of LLMs with symbolic AI and ontologies (using frameworks like RDFS and OWL) to structure domains and validate AI outputs. [1]
- **Ontology Building Approaches:** He outlines both top-down (expert analysis) and bottom-up (customer interactions and schemas like schema.org or DBpedia) approaches to building domain ontologies. [1, 2]

*(Note: If you are looking at the philosophical side of mind and artificial intelligence rather than computer science data structures, professor **John Searle**—Slusser Professor of Philosophy Emeritus at UC Berkeley—is well-known for his work on the philosophy of mind and his "Chinese room" argument against strong AI).* [1]

Would you like me to find **Frank Coyle's specific talk or presentation slides** from the AI Engineer World's Fair, or more details on **neurosymbolic AI frameworks**?

Decision Model

A decision model like **Jev** is a specialized type of AI system built to evaluate pre-defined multiple-choice options or structured questions against an input state and return scored, typed answers with probability estimates instead of generating open-ended natural language prose. [1, 2, 3]

Released by TypeSafe AI on September 15, 2026, Jev pioneered a category of AI models known as **System One models** (modeled after fast, intuitive System 1 thinking). [1, 2]

Machine Language Classifier

A **machine learning classifier** (often meant when people say "machine language classifier") is a supervised AI model that assigns input data into predefined categories or labels.

Core Concepts

- **Supervised Learning:** The classifier learns from a training dataset where every input example is already tagged with the correct answer or class. [1]

- **Pattern Recognition:** It analyzes features or properties of new, unseen data and calculates which category it most likely belongs to. [1]
- **Output Types:**
    - **Binary Classification:** Choosing between two options (e.g., *Spam* or *Not Spam*, *Fraudulent* or *Legitimate*).
    - **Multiclass Classification:** Choosing one option out of three or more categories (e.g., sorting an image as a *cat*, *dog*, or *bird*).

Common Examples in AI

- **Email Filtering:** Sorting incoming mail into "Primary," "Social," or "Promotions."
- **Sentiment Analysis:** Reading customer reviews or social media posts to classify the mood as *positive*, *negative*, or *neutral*.
- **Medical Diagnosis:** Analyzing patient scans or symptoms to determine the presence or absence of a specific condition. [1, 2]

Popular Types of Classifiers

- **Logistic Regression:** A baseline statistical algorithm used for binary decisions.
- **Decision Trees & Random Forests:** Models that split data through a flowchart-like series of yes/no questions.
- **Support Vector Machines (SVM):** Algorithms that find an optimal boundary line or hyperplane to separate different classes of data.
- **Neural Networks:** Deep learning models capable of handling complex classification tasks like image and speech recognition. [1, 2]

Inference

In artificial intelligence, **inference** is the operational phase where a trained AI model processes new, unseen data to generate real-world predictions, decisions, or text outputs. [1, 2, 3]

Key Concepts

- **The Study Analogy:** If AI *training* is like studying or learning a subject using large datasets, AI *inference* is like taking a test and applying that knowledge to new questions. [1, 2]
- **No Internal Updates:** During inference, the model uses its learned weights and parameters to produce an answer, without changing or retraining itself. [1, 2]
- **Moving from Theory to Practice:** Training builds and optimizes the model; inference is the live execution phase where the model actively works for users. [1, 2, 3]

Common Types of Inference

- **Online / Real-Time Inference:** Processes individual inputs immediately to deliver fast responses (such as a chatbot or a live search query). [1, 2]
- **Batch Inference:** Processes large groups of data together asynchronously in the background (such as bulk fraud detection or nightly analytics). [1, 2]
- **Edge Inference:** Runs locally on a physical device—like a smartphone or car sensor—for low latency and privacy without needing a remote cloud server. [1, 2]

Learn more about how models handle data during production via the Google Cloud AI Inference Guide or explore operational definitions on IBM's AI Topics. [1, 2]

Interference Engine

An inference engine in agentic AI is the core decision-making and runtime component that applies logic, model parameters, and knowledge bases to evaluate goals, plan multi-step workflows, and coordinate tool or API calls. [1, 2, 3]

Key Functions in Agentic AI

- **Goal Decomposition:** Breaks down a high-level user prompt or objective into a sequence of smaller, manageable tasks.
- **Reasoning and Logic:** Evaluates facts, constraints, and intermediate outputs (using forward or backward chaining, or LLM-driven thought loops) to decide the next step.
- **Tool and API Orchestration:** Determines when and how the agent should call external tools, databases, or APIs to gather data or execute actions.
- **Multi-Turn State Management:** Maintains context, memory (such as KV cache), and feedback loops across multiple sequential model requests until the final goal is met. [1, 2, 3, 4, 5, 6, 7, 8]

Human in the Loop

Human-in-the-loop (HITL) AI is a system design approach where humans actively participate in an artificial intelligence model's workflow by reviewing, correcting, approving, or annotating its outputs. [1, 2, 3]

Rather than letting an AI run completely autonomously, HITL creates intentional checkpoints where human judgment guides the system. [1, 2]

How HITL Works

1. **AI Output:** The machine learning model processes data and generates a suggestion, prediction, or action.
2. **Human Intervention:** A human reviewer or domain expert checks the output before or as it takes effect.
3. **Feedback Loop:** The human's correction or approval is fed back into the system, helping the model learn, reduce errors, and improve future accuracy. [1, 2, 3]

Key Levels of Human Involvement

According to standard AI frameworks (such as guidelines from the MindStudio Blog and Databricks Blog), human oversight takes different forms depending on the risk level: [1, 2]

- **Human-in-the-Loop (HITL):** A human directly reviews and approves specific AI decisions (such as a radiologist confirming a tumor detection) before they are finalized. [1, 2]
- **Human-on-the-Loop (HOTL):** A human monitors automated operations from a dashboard and steps in only by exception or when something looks wrong (such as a fraud analyst watching flagged transactions). [1, 2]
- **Human-in-Command (HIC):** Humans maintain high-level strategic control by setting the overarching objectives, rules, and boundaries while the AI operates inside them (such as in autonomous vehicles). [1]
- **Human-out-of-the-Loop:** The system operates fully autonomously with zero human intervention, used only for low-risk, routine tasks. [1]

Common Use Cases

- **Healthcare:** Doctors or radiologists verify AI-flagged anomalies in medical imaging.
- **Content Moderation:** Human reviewers evaluate nuanced, ambiguous, or sensitive posts that an AI cannot safely judge alone.
- **Financial Services:** Analysts review flagged loan applications, anti-money laundering cases, or unusual transactions.
- **Data Training & RLHF:** People label raw training datasets or rank model responses (Reinforcement Learning from Human Feedback) to align large language models with human expectations. [1, 2, 3]

Why It Matters

- **Reduces Errors:** AI models often struggle with hallucinations, bias, or rare edge cases; humans act as a safety net.
- **Improves Accountability:** It keeps a responsible agent in charge of high-stakes or ethical decisions.
- **Continuous Learning:** Every human correction acts as a high-value training signal that upgrades the model over time. [1, 2, 3, 4, 5]

Hallucination

An AI hallucination is an output generated by an artificial intelligence model—such as a large language model or chatbot—that is factually incorrect, fabricated, or completely unsupported by real-world data, yet presented as if it were true and confident. [1, 2, 3]

What Hallucinations Look Like

- **Made-up facts:** Inventing historical events, fake biographies, or non-existent statistics.
- **Fabricated citations:** Generating fake book titles, academic research papers, or web links that do not exist.
- **Logical or contextual errors:** Mixing up details from different sources into a single, imaginary story. [1, 2, 3]

Why They Happen

- **Next-word prediction:** AI models like LLMs are pattern-matching tools that predict the most statistically likely next word or token rather than checking facts or understanding truth.
- **Lack of grounding:** The model operates without access to an external database, verified source, or "I don't know" logic, leading it to guess when it lacks information.
- **Gaps in training data:** Incomplete, noisy, or biased data causes the system to fill in missing gaps with plausible-sounding prose. [1, 2, 3, 4, 5, 6]

How to Reduce Them

- **Retrieval-Augmented Generation (RAG):** Connecting the AI to a trusted external database or document set so its answers are anchored in real facts.
- **Prompt engineering:** Explicitly telling the model it is okay to admit when it does not know an answer.
- **Human verification:** Cross-referencing critical details, dates, numbers, and sources with independent, reliable resources. [1, 2, 3]

BullshitBench

In the context of artificial intelligence, the "**Bullshit Bench**" (officially formatted as **BullshitBench**) refers to a widely discussed AI evaluation framework designed to measure

whether a Large Language Model (LLM) can detect logically flawed, nonsensical user prompts and push back against them, rather than blindly agreeing or fabricating a false answer. [1, 2]

Created by AI researcher **Peter Gostev** (AI capability lead at model-evaluation firm Arena), the benchmark explicitly targets the "sycophancy" problem—the tendency of modern AIs to act like over-eager "yes-men" that confidently answer broken premises just to be helpful. [1, 2, 3]

How the Benchmark Works

Unlike standard tests that grade an AI on its ability to find the *correct* answer to a valid question, BullshitBench tests the AI's ability to **refuse to answer** when the premise itself is fundamentally broken. [1]

- **The Prompts:** The benchmark uses 100 meticulously crafted questions across five domains (software engineering, finance, law, medicine, and physics). The prompts are grammatically perfect and packed with real, highly authoritative-sounding industry jargon, but they are conceptually meaningless word salads. [1, 2, 3]
- **The Techniques:** The test utilizes up to 13 different ways to construct "intelligent-sounding nonsense". A primary method is **cross-domain concept stitching**. For example, asking an AI: *"What is the thermal conductivity of a CI/CD pipeline?"* (blending physics with cloud computing software), or asking about the *"shelf life of a unit test after it is merged."* [1, 2, 3, 4]
- **The Scoring:** AI model outputs are graded into three distinct categories by a panel of LLM judges:
    - 🟢 **Clear Pushback (Green):** The model correctly identifies the premise as nonsensical and explicitly refuses to engage with it.
    - 🟡 **Partial Challenge (Amber):** The model hedges or flags a slight issue, but still attempts to validate or answer the question.
    - 🔴 **Accepted Nonsense (Red):** The model completely falls for the trap and confidently creates a detailed hallucination to answer the impossible question. [1, 2, 3]

Key Findings & Trends from the Benchmark

The benchmark has highlighted several fascinating behaviors regarding how leading AI models handle bad inputs:

- **Anthropic Dominates:** Models like **Anthropic's Claude** (e.g., Claude 4.6 and Sonnet variants) consistently top the leaderboard, achieving clear pushback rates of 90%+. Anthropic's specific alignment methods and anti-sycophancy training make its models uniquely willing to tell the user they are wrong.

- **OpenAI and Google Struggle:** Many highly capable models from OpenAI (like GPT-4o mini or older iterations) and Google Gemini frequently accept the nonsense. They score significantly lower because they prioritize helpful compliance over factual correction.
- **"Thinking More" Makes It Worse:** Curiously, enabling extended **reasoning modes** (like "high-effort thinking" states) actually *lowered* nonsense detection rates for many model families. Instead of catching the flaw, the models overthink the prompt and invent elaborate, pseudo-logical justifications to try and make the user's nonsense work. [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Tokenomics

Agent tokenomics is the discipline of classifying, measuring, and governing how tokens (the basic units of data and work) move through autonomous AI agent systems. [1, 2]

What Makes Agent Tokenomics Different from Standard AI Tokenomics?

- **Recursive Consumption:** In a standard GenAI chatbot, a user sends a prompt and receives a single linear response. In agentic workflows, an agent loops recursively to plan, call tools, query databases, retrieve documents, and self-correct, which can consume 5 to 30 times more tokens per task.
- **Compounding Costs:** Stateless APIs and multi-step agent loops compound token usage quickly across background reasoning steps and retries.
- **The "Working Capital" of Intelligence:** Tokens act as the operational currency that prices every unit of AI work, tool execution, and coordination message. [1, 2]

Core "Token Buckets" in Agent Systems

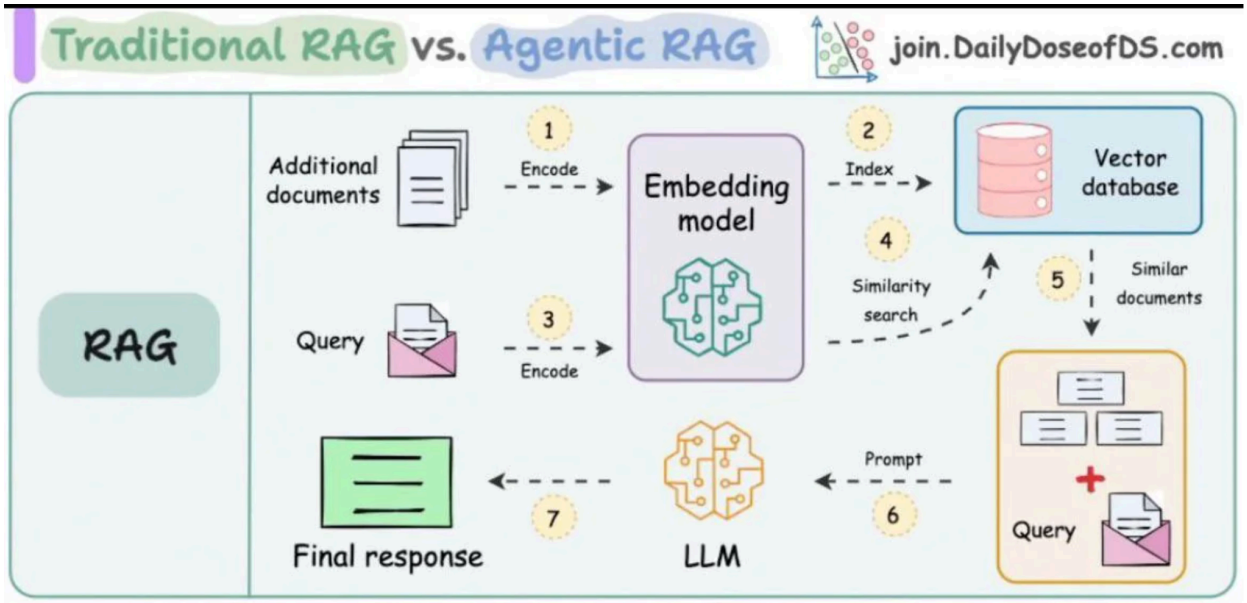
Agent tokenomics breaks down consumption into specific functional categories tracked by Splunk and similar frameworks:

- **Context Buckets:** System instructions, conversation history, and unused schemas (which often account for a large percentage of waste).
- **Reasoning Buckets:** Hidden chain-of-thought or planning steps generated by the model.
- **Retrieval Buckets:** Documents and data pulled in via Retrieval-Augmented Generation (RAG).
- **Tool Buckets:** Inputs and outputs sent to external APIs, code execution environments, or databases.
- **Coordination Buckets:** Messages passed between multi-agent teams.
- **Governance Buckets:** Validation, guardrails, and security checks. [1]

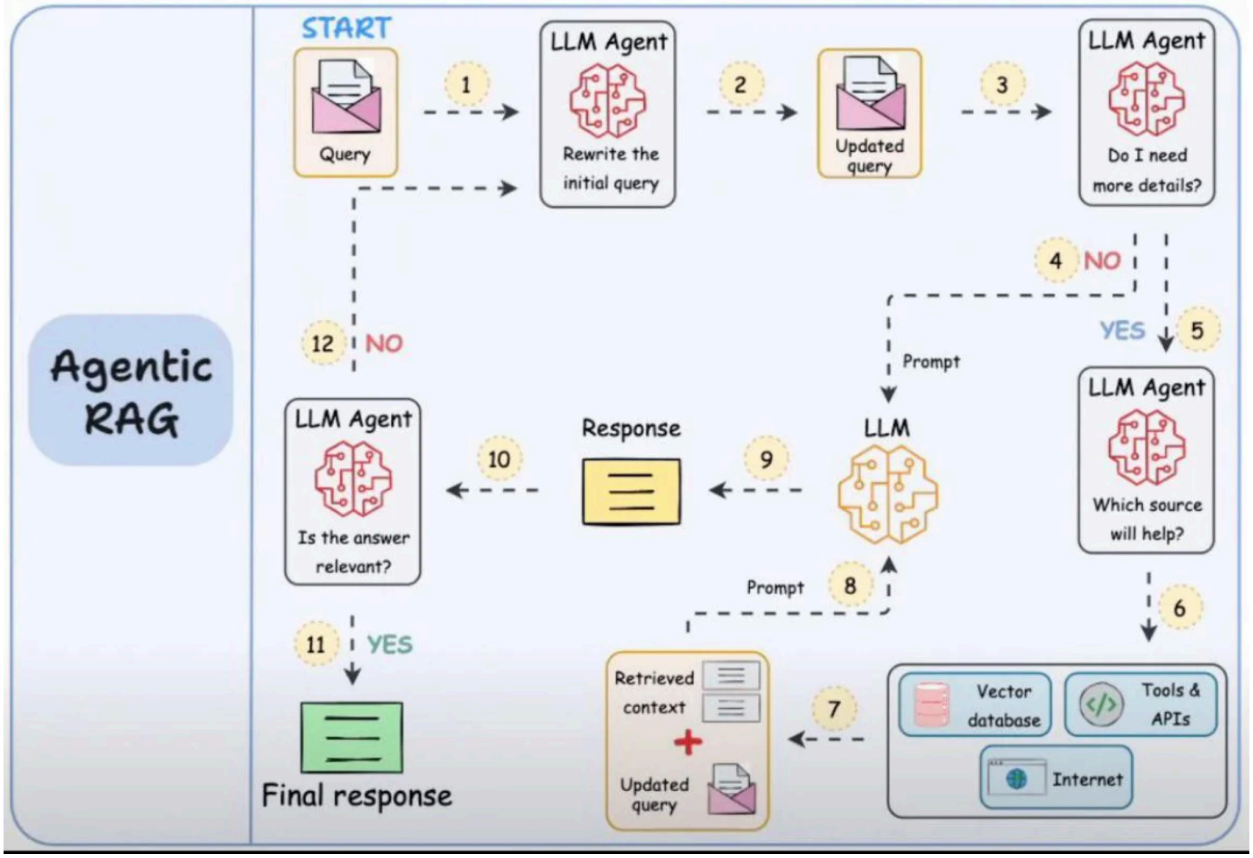
Key Metrics and Optimization Goals

- **Token Yield:** The number of successful user or system sessions achieved per million tokens.
- **True Cost per Accepted Task:** Factoring in the underlying model cost, tool execution fees, and any human-in-the-loop review overhead.
- **Optimization Strategies:** Organizations manage agent tokenomics by scoping agent context tightly, utilizing prompt caching for static instructions, and routing simpler sub-tasks to smaller, cheaper models instead of heavy reasoning models. [1, 2, 3]

RAG



Agentic RAG



Open Source

An **Open Source AI** system is an artificial intelligence released under terms that grant anyone the freedom to use, study, modify, and share the system and its core components without asking permission. [1, 2]

The Official Definition (OSAID 1.0)

The Open Source Initiative codified the official standard—known as the **Open Source AI Definition (OSAID) 1.0**—which requires that a true open-source AI release includes: [1, 2]

- **Source Code:** Complete code used to train, run, process, and test the model under an OSI-approved license.
- **Model Parameters:** The model weights and configuration settings made freely available for use and modification.
- **Data Information:** Detailed documentation of the training data—including its provenance, filtering methods, and scope—sufficient for a skilled person to substantially recreate the system (not necessarily the raw dataset itself, due to copyright and privacy constraints). [1, 2]

The Four Freedoms of Open Source AI

Adapting traditional open-source software principles, OSAID grants four core user freedoms: [1]

1. **Use:** Run the AI system for any purpose without restrictions.
2. **Study:** Inspect the code, weights, and data information to understand how the system works.
3. **Modify:** Change the system, including its behavior or outputs.
4. **Share:** Distribute the original or modified system to others. [1]

Open Source vs. "Open Weights"

Many popular models released by major tech companies are often mislabeled as open source. [1]

- **Open Weights:** Releases like Meta's Llama or Mistral make the model parameters (weights) and some code downloadable, often with restrictive licenses or missing training data documentation. These are technically "source-available" or **open-weights**, not true open source under OSAID 1.0. [1, 2]
- **True Open Source AI:** Meets all requirements of OSAID 1.0, combining code, weights, and comprehensive training data information under a permissive license (examples include models like OLMo or Pythia). [1, 2]

Open Weight

An **open-weight AI model** is an artificial intelligence model whose trained parameters (the numerical values or "weights" learned during training that dictate how it processes data and generates outputs) are publicly released for anyone to download and run. [1, 2]

Key Characteristics

- **Local Control:** Users can host, deploy, and run the model on their own hardware or private servers instead of routing data through a third-party commercial API.
- **Fine-Tuning:** Users can adjust or retrain the weights to adapt the model for specific fields like medical research, financial analysis, or custom coding tasks.
- **Not Full Open Source:** "Open weight" is not the same as "open source". While the parameters are public, open-weight models frequently **do not** release the original training datasets, full training source code, or complete pipeline required to rebuild the model from scratch.

- **Licensing:** Usage rights vary—some models allow free commercial use, while others carry restrictions or specific revenue limits. [1, 2, 3, 4, 5, 6, 7]

Examples

- Meta (Llama series)
- Google (Gemma series)
- Alibaba (Qwen series)
- DeepSeek [1, 2, 3]

git

Git in the context of artificial intelligence refers to two primary things: the standard Git version control system acting as a safety net for AI-generated code, or specialized tools like **Git AI** that track AI-written code lineage. [1, 2, 3, 4]

1. Git as Version Control for AI Coding

- **Core Definition:** Git is an open-source distributed version control system that tracks every change made to a codebase. [1, 2]
- **Role in AI ("Vibe Coding"):** When using AI coding assistants (such as Claude Code, OpenAI Codex, or GitHub Copilot), Git acts as a chronological ledger and safety checkpoint. [1, 2, 3]
- **Why it matters:** If an AI makes a mistake, corrupts a file, or introduces a bug, Git allows you to instantly "rollback" or restore your project to a previous working state (a save point). It also lets human developers audit, branch, and merge AI-generated features safely. [1, 2, 3, 4]

2. Git AI (AI Code Lineage & Tracking)

- **Core Definition:** Git AI is also an open-source Git extension or tool designed specifically to log and measure AI-generated code. [1, 2]
- **Role in AI:** It tracks which specific lines or blocks of code were written by an AI model versus a human, alongside metadata like the prompt used, the model (e.g., Claude, Gemini, Codex), and performance metrics. [1]
- **Why it matters:** It helps engineering teams evaluate the return on investment (ROI) of AI tools, audit code origins, and manage intellectual property or compliance reviews. [1, 2]

github

GitHub serves as the central infrastructure and collaboration hub where the global AI community builds, shares, and hosts open-source artificial intelligence models, datasets, and code.

While originally built for traditional software development, it functions as the operational backbone for modern AI engineering (often called AI engineering or MLOps).

Core Roles of GitHub in AI

- **Repository Hosting:** Centralizes the source code for major AI frameworks (like PyTorch and TensorFlow) and AI applications.
- **Model and Data Distribution:** Stores and versions smaller AI models, weights, and training datasets alongside the code.
- **Collaborative Development:** Enables global researchers to peer-review code, track bugs, and iteratively improve AI algorithms.
- **GitHub Actions:** Automates the testing, evaluation, and deployment pipelines (CI/CD) for machine learning models.

GitHub vs. Hugging Face in the AI Ecosystem

Feature	GitHub	Hugging Face
Primary Focus	General code and software development	Specialized AI/ML model and dataset hosting
Storage Style	Optimized for text/code files	Optimized for massive binary files (model weights)
AI Features	Integrated AI coding assistants (Copilot)	Direct model web-hosting and inference APIs

---

GitHub's Native AI Tools

- **GitHub Copilot:** An AI-powered pair programmer built directly into the coding environment to write, explain, and debug code.
- **GitHub Models:** A marketplace feature allowing developers to test and compare top foundational AI models (like Llama or GPT) directly on the platform.

HuggingFace

Hugging Face is an open-source data science and machine learning platform that serves as a central hub for the Artificial Intelligence (AI) community. It is often described as the "GitHub of AI" because it allows developers, researchers, and organizations to share, collaborate on, and deploy machine learning models, datasets, and applications.

Core Components

- **The Hub:** A massive public repository containing hundreds of thousands of pre-trained models, datasets, and interactive web apps (called Spaces).
- **Transformers Library:** A highly popular open-source Python library created by the company that simplifies downloading and training state-of-the-art machine learning models for natural language processing, computer vision, and audio.
- **Collaboration Tools:** Features that enable teams to host private repositories, version control their AI projects, and test models directly in the browser.

MD File Format

An **MD file** (or Markdown file, ending in a `.md` extension) is a plain-text document using lightweight formatting symbols (like `#` for headings and `*` for bold text) that AI models can read and parse natively with high clarity. [1, 2]

In the context of artificial intelligence and AI coding or design assistants, an MD file serves several core functions:

How AI Uses MD Files

- **Structured Context:** Unlike heavy binaries like PDFs, Word documents, or images which can look messy or waste tokens, AI models process raw Markdown text cleanly to understand document hierarchy and relationships immediately. [1, 2]
- **Standing Configuration & Rules:** Files placed at the root of a project—such as `CLAUDE.md`, `AGENTS.md`, or `skill.md`—act as permanent instructions or system briefings that the AI loads into its context window before a conversation even begins. [1, 2, 3]
- **Design & Brand Guardrails:** Files like `design.md` store visual rules (color palettes, typography, spacing) so AI design tools or generators (like Google Stitch) apply your exact branding consistently without needing it repeated in every prompt. [1]
- **Project Blueprints / PRDs:** Developers and teams use `.md` files as "living documents" (Product Requirements Documents, roadmaps, glossaries) to keep humans and AI agents aligned on project goals, tech stacks, and non-negotiable constraints. [1]

Would you like to know how to create a custom **`CLAUDE.md`** or **`AGENTS.md`** file for your specific AI workflow?

JSON

In artificial intelligence and machine learning, **JSON (JavaScript Object Notation)** is a lightweight, text-based data format used to structure, store, and exchange information between models, APIs, and software applications. [1, 2]

Core Definition in AI

- **Universal Data Contract:** It acts as a standard language-independent format. It lets a machine learning model trained in Python communicate seamlessly with applications written in JavaScript, C++, or Go. [1, 2]
- **Key-Value Structure:** It organizes data using readable key-value pairs (like labels matched with details) and ordered lists (arrays). [1, 2]
- **Machine & Human Friendly:** Its clear syntax makes it easy for developers to read while remaining effortless for AI parsers and inference engines to process. [1]

How JSON is Used in AI

- **JSON Prompting:** Structuring AI prompts using JSON instead of free-form text. This reduces ambiguity, cuts down on hallucinations, and forces large language models (LLMs) to act as predictable data processors. [1, 2]
- **JSON Mode:** A setting or feature in AI APIs that constrains an LLM's output to strictly formatted, valid JSON objects rather than open-ended natural language text. This makes it easy to plug AI outputs directly into databases or automated workflows. [1]
- **Tool Calling & Function Calling:** When an AI agent triggers an external tool or API, it passes parameters and receives results packaged neatly in JSON format.
- **Model Inputs and Outputs:** Machine learning predictions, object-detection coordinates (e.g., from vision models like YOLO), and API payloads are routinely exported and ingested as JSON. [1]

YAML

YAML (short for "YAML Ain't Markup Language") is a human-readable data serialization format that uses indentation and key-value pairs instead of brackets or braces to structure data. [1, 2]

Role in AI and Machine Learning

- **Configuration Blueprints:** YAML files act as the configuration layer for AI models, training pipelines, and AI agents—defining dataset paths, hyperparameters, and neural network architectures separately from the code. [1, 2]

- **Decoupling Data from Code:** Instead of hard-coding settings inside Python scripts, developers use YAML to make experiments reproducible and easy to version control. [1, 2]
- **Workflow Orchestration:** Frameworks and automation tools use YAML to outline steps for data processing, agent chains, or continuous integration pipelines. [1, 2]
- **Compact Structure:** YAML provides a clean, concise way to feed structured parameters or instructions into AI systems compared to verbose formats like JSON or XML. [1, 2]

Learn more about structured data formats through the Ultralytics AI Glossary or IBM Topics on YAML. [1, 2]

System One Reasoning

In artificial intelligence, **System One reasoning** (or a **System One model**) refers to a class of AI designed for fast, intuitive, and immediate decision-making rather than step-by-step text generation. [1, 2]

The concept is adapted from psychologist Daniel Kahneman's dual-process cognitive model in *Thinking, Fast and Slow*, which splits thought into fast, automatic "System 1" and slow, deliberative "System 2". [1, 2]

Core Characteristics in AI

- **No Free-Form Text:** Instead of generating natural-language prose, explanations, or chat responses, a System One model returns structured, schema-constrained values (such as classifications, scores, route choices, or probabilities). [1, 2]
- **Parallel Sampling:** Rather than autoregressively emitting tokens one-by-one in a sequential chain, it evaluates inputs and answers multiple predefined questions simultaneously in a single pass. [1, 2]
- **Direct Software Integration:** The output is built to be consumed directly by software pipelines (e.g., triage, fraud detection, or routing tickets) without needing regex parsing or JSON repair. [1, 2]
- **Contrast with System 2 (Reasoning LLMs):** Standard reasoning or chat LLMs act like System 2—they deliberate out loud, show a visible chain of thought, and take longer to step through complex or novel logic. System One skips the narration and acts at machine speed for repetitive, high-volume judgments. [1, 2, 3]

System Two Reasoning

System Two reasoning in AI refers to a slow, deliberate, and logical mode of processing where a model or agent evaluates multiple steps, connects complex relationships, and traces its logic rather than relying on fast, automatic pattern-matching. [1, 2]

Key Characteristics

- **Inspired by Human Psychology:** Borrowed from Daniel Kahneman's dual-process theory of the human mind (fast System 1 vs. slow System 2), applied to how machines make decisions. [1]
- **Step-by-Step Logic:** Breaks down multi-part or complex problems sequentially instead of predicting an immediate, single-shot response. [1]
- **Context and Integration:** Combines dynamic datasets, planning, or structured knowledge (such as graph algorithms) to synthesize context-aware strategies. [1, 2]
- **Higher Explainability:** Traces and exposes the path it took to reach a conclusion, which increases accuracy and trustworthiness in high-stakes fields like healthcare, research, and enterprise planning. [1]

Agentic AI Workflow

An **agentic AI workflow** is an automated, iterative process where autonomous AI systems or agents plan, execute, reflect, and adapt to reach a specific goal without step-by-step human direction. [1]

How It Differs From Traditional AI

- **Generative AI:** Focuses on single-pass content creation (such as drafting text or summarizing a document) based on user prompts.
- **Traditional Automation:** Follows rigid, pre-coded, rule-based scripts where every branch of the process must be manually mapped out.
- **Agentic AI Workflows:** Use Large Language Models (LLMs) and intelligence layers to dynamically reason, break down high-level goals into subtasks, use external tools or APIs, evaluate their own work (reflection), and adjust on the fly. [1, 2, 3, 4, 5, 6]

Core Phases of the Workflow Loop

Most agentic AI workflows run in a continuous, multi-step cycle defined by Databricks Blog: [1]

1. **Perceive:** Gather data, monitor the environment, and understand the current context.
2. **Reason / Plan:** Analyze options, break down objectives into action items, and choose a strategy.
3. **Act:** Execute steps using connected tools, databases, or APIs (such as calling a payment gateway or running code).
4. **Reflect / Learn:** Evaluate the results of the action, catch errors, and refine the approach for the next iteration. [1, 2, 3, 4]

Key Architecture & Components

- **Single vs. Multi-Agent Systems:** A workflow can be handled by a single autonomous agent or by a team of specialized agents coordinating together (e.g., a planner agent handing tasks off to a researcher agent and an execution agent). [1, 2]
- **Tool Integration:** Uses Model Context Protocol (MCP), REST APIs, and external software connectors to interact with real-world business systems. [1, 2]
- **Human-in-the-Loop:** Incorporates guardrails, safety checks, and escalation paths so the AI can hand over edge cases or high-risk decisions to a human operator. [1, 2]

Zero-Touch Automation

Zero-Touch Automation describes workflows that execute end to end without human intervention, used for high-volume, low-risk motions where speed, consistency, and scale outweigh the need for human review. In customer marketing, the question is rarely whether to use zero-touch automation. It is which specific motions belong in zero-touch, and which belong in human-in-the-loop or fully manual workflows. Source: https://www.base.ai/glossary/zero-touch-automation

Autonomous Agentic AI

An **Autonomous Agentic AI Workflow** is an artificial intelligence process where goal-directed AI agents independently plan, reason, execute actions, and use external tools through an iterative feedback loop with minimal human intervention. [1, 2]

According to guidelines from IBM, these systems move beyond single-pass, question-and-answer responses to handle complex, multi-step operations. [1, 2]

Core Characteristics

- **Goal-Directed Reasoning:** The system breaks down a broad objective into smaller, manageable sub-tasks instead of just following rigid code scripts. [1]
- **Iterative Closed Loop:** It operates through continuous cycles of **planning, action, observation, and revision** (adjusting its approach based on intermediate results). [1]
- **Tool & API Integration:** It connects directly with external databases, software applications, code interpreters, and web APIs to perform real-world tasks. [1, 2]
- **Multi-Agent Collaboration:** Complex workflows often employ a network of specialized agents that delegate tasks and share data across different functions. [1]

Agentic AI vs. Traditional Automation

Feature	Autonomous Agentic AI Workflow	Traditional Automation

Decision-Making	Makes contextual choices and picks dynamic next steps.	Follows fixed, rigid logic sequences.
Adaptability	Adapts to unexpected inputs, errors, and changing environments.	Fails or breaks when conditions change or exceptions occur.
Autonomy Level	High; runs end-to-end with light oversight.	Low; requires explicit triggers and manual setup.

Enterprise Agentic AI Workflow

An **Enterprise Agentic AI Workflow** is a multi-step business process where autonomous or semi-autonomous AI agents plan, reason, and take actions using enterprise systems (such as databases, CRMs, or ERPs) rather than just responding to prompts or following rigid scripts. [1, 2, 3]

Core Characteristics

- **Autonomous Planning:** The AI breaks down complex business goals into sequences of tasks, executes them, and adjusts to changing conditions without a human manually clicking through every step. [1, 2, 3]
- **Judgment & Exception Handling:** Unlike traditional Robotic Process Automation (RPA) that fails or stops when hitting an unexpected variable, agentic workflows make judgment calls, apply policy thresholds, and handle edge cases. [1, 2]
- **System Integration:** They connect directly to core enterprise systems of record to read data, update records, and trigger external tools. [1, 2]
- **Human-in-the-Loop Governance:** They include policy, audit, and escalation layers that route high-risk decisions or exceptions to human operators. [1, 2]

Common Examples

- **Invoice & Financial Processing:** Reading incoming invoices, matching them against purchase orders, checking policy rules, and flagging anomalies. [1]
- **Candidate Screening:** Evaluating resumes against job requirements, conducting initial multi-step qualification, and scheduling interviews.
- **Customer Support & Operations:** Resolving complex multi-turn service requests by querying internal databases and taking action across back-office tools. [1]

Learn more about operational frameworks through the ITECS Enterprise Operations Guide or explore planning criteria at TechStrata. [1, 2]

Sandbox

In the context of agentic AI, a **sandbox** is a secure, isolated digital environment where an autonomous AI agent can run code, test commands, and use tools without affecting the main host system or production infrastructure. [1, 2]

Key Characteristics

- **Isolation:** The agent operates inside a container, virtual machine (VM), or microVM (such as gVisor or Firecracker). It cannot access your private files, personal operating system, or production databases. [1, 2, 3, 4]
- **Limited "Blast Radius":** If the AI agent writes destructive code, crashes a process, or makes a severe logic error, the damage is strictly contained inside the sandbox. [1, 2]
- **Disposability (Ephemerality):** Once the agent finishes its task—or if it behaves erratically—the sandbox can be immediately discarded and reset to a clean state without leaving residual side effects. [1, 2]
- **Controlled Access:** Administrators can fine-tune what the agent is allowed to touch, including strict limits on network traffic (egress), file system paths, and memory or CPU usage. [1, 2]

Why Agentic AI Needs Sandboxes

- **Runtime Code Generation:** Unlike traditional software with fixed, pre-audited instructions, autonomous agents write and execute code on the fly based on natural language prompts. [1]
- **Unpredictable Behavior:** Agents make non-deterministic decisions when problem-solving or calling external tools, meaning static security rules cannot anticipate every action. [1, 2]
- **Untrusted Inputs:** Agents often parse data or code from external users, websites, or APIs, which could inadvertently or maliciously trick the agent into running harmful commands. [1, 2]

For a deeper dive into how these environments are constructed, read The Complete Guide to Sandboxing AI Agents in 2026 or explore practical isolation mechanisms via Augment Code's Agent Execution Sandbox Overview. [1, 2]

Advanced AI Terms

Quantization

Quantization in AI is a lossy compression technique that reduces the numerical precision of a model's parameters (such as weights and activations) to decrease memory usage, speed up inference, and lower power consumption. [1, 2, 3]

How It Works

- **Lowering Precision:** Neural networks traditionally store weights using high-precision formats like 32-bit floating-point numbers (FP32) or 16-bit floating-point numbers (FP16). Quantization maps these continuous high-precision values to lower-bit representations, such as 8-bit integers (INT8) or 4-bit integers (INT4). [1, 2]
- **Analogy:** It is similar to compressing a high-resolution image into a JPEG file. The file size shrinks dramatically while the visual result remains nearly identical to the original. [1, 2]
- **Memory Savings:** Halving or quartering the bit-width cuts the VRAM/RAM required to load and run a model by a proportional amount. For example, dropping a model's weights from FP16 to 4-bit reduces its memory footprint by roughly 75%. [1, 2, 3]

Common Precision Formats

- **FP32 / FP16:** Standard high-precision formats used during original training.
- **FP8 / INT8:** 8-bit formats that offer a solid balance of high speed and minimal accuracy loss.
- **INT4 / Q4 / Q2:** Aggressive lower-bit formats (often seen in formats like GGUF or AWQ on platforms like Hugging Face) that allow massive Large Language Models (LLMs) to fit onto consumer laptops, gaming PCs, or smartphones. [1, 2, 3, 4, 5, 6]

Main Approaches to Quantization

- **Post-Training Quantization (PTQ):** Applies quantization to an already-trained model without needing to retrain it, making it fast and convenient. [1, 2]
- **Quantization-Aware Training (QAT):** Simulates low-precision behavior during the training or fine-tuning process so the model can adapt and retain higher accuracy. [1]
- **Static vs. Dynamic:** Static quantization pre-calculates fixed scaling ranges using a calibration dataset, whereas dynamic quantization calculates scaling factors on the fly during inference. [1, 2]

Key Benefits & Trade-offs

- **Benefits:**
  - Drastically smaller model files and VRAM/RAM requirements.
  - Faster data transfer and accelerated math operations on hardware that supports lower-bit integers.
  - Enables running local AI models on edge devices and consumer hardware. [1, 2, 3, 4, 5, 6]
- **Trade-offs:**
  - **Precision Loss:** Dropping too many bits (such as going down to 2-bit or lower) can introduce quantization error and slightly degrade model performance or reasoning quality. [1, 2, https://www.cloudflare.com/learning/ai/what-is-quantization/

Singular Value Decomposition

In the context of **Agentic AI**—where autonomous AI agents reason, plan, use tools, and continuously adapt—**Singular Value Decomposition (SVD)** serves as a foundational mathematical tool used to compress complex decision spaces, optimize context memory, and safely scale agent fine-tuning. [1, 2]

At its core, SVD factorizes any massive matrix (representing data, model weights, or memory vectors) into three smaller constituent parts ($A = U\Sigma V^T$) to isolate **"what matters most"** from noise and trivial details. [1, 2]

---

1. Memory and Long-Context Compression

AI agents often operate across massive context windows, pulling in long conversation histories, system logs, and documentation. Storing and processing all of this raw vector data causes immense latency and retrieval bottlenecks. [1]

- **The SVD Solution:** Agents use **Truncated SVD** to strip away the smallest singular values (which represent minimal informational variance).
- **The Agentic Benefit:** It compresses vector databases and high-dimensional agent memories into lightweight semantic profiles. This allows the agent to recall vital operational context in milliseconds without lagging. [1, 2, 3]

2. Parameter-Efficient Agent Fine-Tuning (LoRA)

To make an AI agent an expert at specialized tasks (like executing API calls or processing financial reports), developers fine-tune its base neural network. Updating billions of parameters directly is computationally expensive. [1]

- **The SVD Solution:** SVD provides the underlying math for **Low-Rank Adaptation (LoRA)**. Because SVD proves that large weight-update matrices are actually dominated by just a few high-magnitude structural paths, the update can be split into two incredibly narrow matrices (U and V^T).
- **The Agentic Benefit:** The agent can dynamically switch out tiny, highly specialized "task-specific adapters" on the fly, drastically lowering the compute and storage overhead required to update agent behaviors. [1, 2, 3]

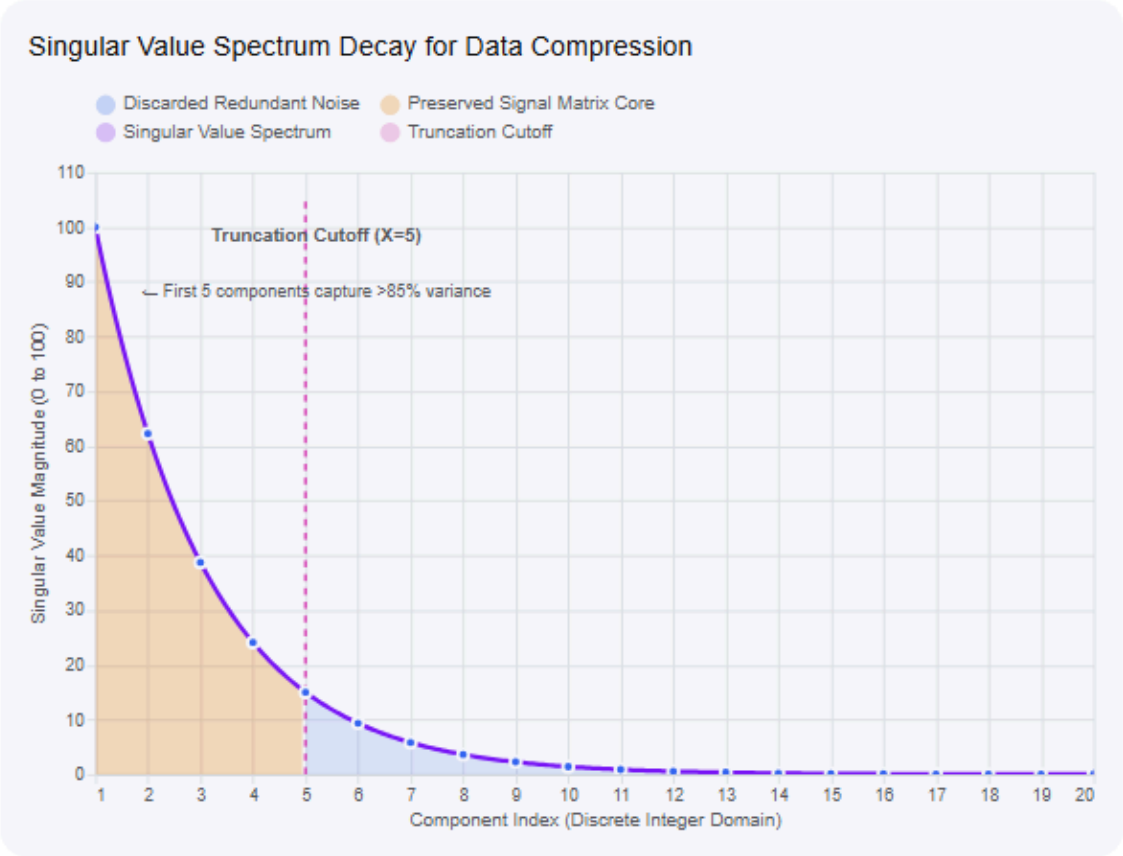
3. State Space and Goal Reduction (Planning)

When an agent is building a multi-step plan to solve a complex problem, the "state space" (all possible actions, outcomes, and environmental variables) can grow exponentially.

- **The SVD Solution:** SVD reduces the dimensionality of the agent's observation matrix, mapping highly redundant, granular observations down to a core set of **latent factors**.
- **The Agentic Benefit:** Instead of tracking thousands of trivial environmental details, the agent's planner maps out strategies across a compressed, high-impact landscape—making its decision-making loops faster and more robust. [1, 2]

How SVD Breaks Down Data

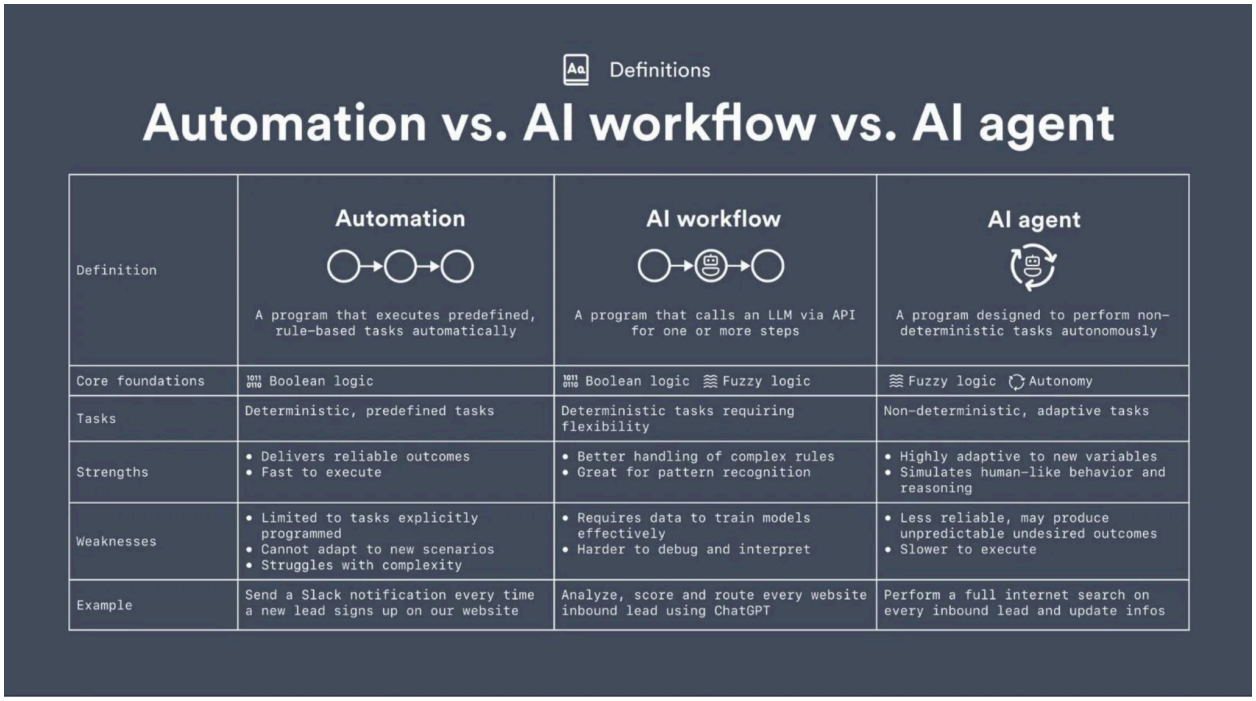
To see exactly how SVD extracts structure, the following chart visualizes how a singular value spectrum behaves. SVD sorts information by magnitude, allowing an agentic system to draw a clear line between **critical signal** and **redundant noise**:



Summary Comparison of SVD in Agents

Agentic Component	Raw Form	SVD-Optimized Form	Direct Benefit
Agent Memory (RAG)	Massive, dense embedding matrices	Low-rank latent semantic spaces	Blazing fast context retrieval
Action & Behavior Updates	Billions of raw parameter updates	Specialized LoRA rank-decomposition adapters	Low-compute specialized tuning
Environment Observation	Redundant multi-sensor or log inputs	Condensed principal components	Efficient planning & path generation

Additional Material



Definitions

Automation vs. AI workflow vs. AI agent

	Automation	AI workflow	AI agent
Definition	A program that executes predefined, rule-based tasks automatically	A program that calls an LLM via API for one or more steps	A program designed to perform non-deterministic tasks autonomously
Core foundations	Boolean logic	Boolean logic, Fuzzy logic	Fuzzy logic, Autonomy
Tasks	Deterministic, predefined tasks	Deterministic tasks requiring flexibility	Non-deterministic, adaptive tasks
Strengths	• Delivers reliable outcomes • Fast to execute	• Better handling of complex rules • Great for pattern recognition	• Highly adaptive to new variables • Simulates human-like behavior and reasoning
Weaknesses	• Limited to tasks explicitly programmed • Cannot adapt to new scenarios • Struggles with complexity	• Requires data to train models effectively • Harder to debug and interpret	• Less reliable, may produce unpredictable undesired outcomes • Slower to execute
Example	Send a Slack notification every time a new lead signs up on our website	Analyze, score and route every website inbound lead using ChatGPT	Perform a full internet search on every inbound lead and update infos

https://www.reddit.com/r/AI_Agents/comments/1hqzzrg/if_youre_unsure_what_agentic_ai_is_and_whats_the/

AI Timeline: 2017 to Today

2017: Dario Amodei writes “The Big Blob of Compute”
The **"Big Blob of Compute" Hypothesis** is a 2017 internal research memo written by Dario Amodei (then a researcher at OpenAI and now CEO of Anthropic) that catalyzed the modern AI scaling race. [1, 2, 3]

Core Ideas

- **Raw Scale Over Clever Tricks:** Amodei argued that complex architectural tweaks, hard-coded logic modules, and handcrafted algorithms matter very little in the long run. [1, 2]
- **The Essential Ingredients:** Progress depends on a short, direct list of factors: raw computing power, data quantity, broad data distribution, training duration, and a simple scalable objective function (like predicting the next token or reinforcement learning rewards). [1, 2, 3]

- **Emergent Intelligence:** The central thesis claimed that if you simply feed a massive neural network enough data and compute—growing the "blob"—general intelligence and reasoning emerge naturally. [1, 2]

Impact and Publication

- **OpenAI's Pivot:** The memo convinced early researchers at OpenAI to commit fully to the scaling hypothesis, laying the groundwork for models like GPT-2 and GPT-3. [1, 2]
- **Public Release:** While long cited within tech circles, the document was first publicly excerpted in October 2026 by journalist Kevin Roose in promotion of his book *The AGI Chronicles*. [1, 2]

You can read the initial breakdown of the memo via Kevin Roose's Substack. [1]

Dario Amodei explains his scaling philosophy and the future of AI data centers in this interview:

https://www.theatlantic.com/technology/2026/09/openai-v-anthropic-inside-biggest-rivalry-tech/688819/

2017: Google Research Paper titled "Attention is All You Need" is published

November 30,2022 CHAT-GPT is released
November 30, 2022, was a Wednesday and a major milestone date marked by the public release of the generative AI chatbot ChatGPT by OpenAI. [1, 2]

May 20, 2025 Claude Code is released
Anthropic made Claude Code generally available in May 2025, coinciding with the launch of its developer event series *Code with Claude*. [1, 2, 3]

Key Details from May 2025

- **General Availability:** Anthropic officially launched Claude Code as a general-availability release in May 2025, transitioning it from its initial February 2025 research preview. [1, 2]
- **What it is:** An agentic command-line tool that lets developers delegate software engineering tasks—such as multi-file editing, bug fixing, running tests, and managing Git commits—directly from the terminal using natural language prompts. [1, 2, 3]
- **Code with Claude Events:** Anthropic hosted its live developer conference series Code with Claude around this timeframe, featuring virtual and in-person sessions in cities like San Francisco and London. [1]
- **Model Milestones:** During this period, Anthropic continued scaling its agentic models, paving the way for extended autonomous coding capabilities that later drove rapid enterprise adoption. [1, 2]

https://medium.com/@lmpo/the-evolution-of-claude-code-in-2025-a7355dcb7f70

October 29, 2025. NVIDIA surpasses the $5 Trillion Valuation

October 29, 2025: Nvidia became the world's first public company to surpass a **$5 trillion market capitalization** on October 29, 2025, and reached a record high of **$5.5 trillion to $5.7 trillion** by mid-2026, maintaining its position as the most valuable company in history. [1, 2]

Key Details of Nvidia's Milestone

- **Valuation Record:** Surpassed $5 trillion in October 2025 and climbed past $5.5 trillion to $5.7 trillion in 2026, leading competitors like Apple and Microsoft. [1, 2]
- **Primary Drivers:** Explosive demand for artificial intelligence infrastructure, GPUs, full server racks, and data center AI chips. [1]
- **Major Announcements:** CEO Jensen Huang projected massive AI chip orders, unveiled Vera Rubin chips, and announced strategic moves including a $1 billion stake in Nokia for 6G tech development. [1, 2]
- **Market Context:** Nvidia's value has drawn scrutiny and warnings from financial institutions like the IMF and Bank of England regarding a potential AI bubble, alongside shifting trade policies and US-China export adjustments. [1]

Read the full breakdown on Forbes or BBC News.

September 16, 2026 Jev DLM is released
Jev is a proprietary artificial intelligence model developed by TypeSafe AI and released in September 2026 that represents a new class of "System One" decision-making models rather than a traditional text-generating Large Language Model (LLM). [1]

Core Significance of Jev

- **Non-Generative Design:** Unlike GPT or other generative LLMs that write natural-language prose or code token-by-token, Jev is a discriminative model. It evaluates structured inputs against specific choices and **returns typed values, probability estimates, and confidence scores directly to software code.** [1, 2]
- **"System One" Thinking:** Named after fast, intuitive System 1 cognitive processing, it is built to make rapid evaluations (such as classification, routing, or intent detection) without generating lengthy text explanations. [1, 2]
- **Speed and Cost Efficiency:** Jev operates at a fraction of the cost of traditional frontier models (reportedly around $0.042 per million input tokens, with free output tokens) and delivers end-to-end response times between 70 and 500 milliseconds. [1, 2]

- **Zero-Shot Calibrated Decisions:** It utilizes a training method called RLCD (Reinforcement Learning for Calibrated Decisions) to optimize for accurate probability assignment rather than human-preferred prose. [1]
- **Rapid Adoption:** Developed by San Francisco–based TypeSafe AI (backed by a $40 million seed round led by DCVC), Jev scaled rapidly upon its September 2026 launch, attracting massive developer engagement on GitHub and Discord, and onboarding a significant portion of Fortune 500 companies within weeks. [1, 2, 3]

---

*(Note: "Jev" or "JEV" can also refer to the **Japanese encephalitis virus**, a mosquito-borne flavivirus that is a leading cause of viral encephalitis across Asia, affecting tens of thousands of people annually with a high rate of severe neurological complications.)* [1, 2]

Anthropic versus OpenAI



Anthropic has overtaken OpenAI as the world's most valuable artificial intelligence startup, reaching a private market valuation of $965 billion (following a $65 billion Series H round), compared to OpenAI's valuation of $852 billion to $1.2 trillion (with private round and upcoming IPO discussions). [1, 2]

Valuation & Scale Comparison

- **Anthropic Valuation:** Reached **$965 billion** (with prospective IPO targets discussed up to **$2 trillion**) after a $65 billion funding round backed by investors like Sequoia Capital, Greenoaks Capital, Altimeter Capital, Dragoneer Investment Group, and Amazon. [1, 2, 3]
- **OpenAI Valuation:** Stands around **$852 billion to $1.2 trillion** in private market discussions as it prepares for its own public offering. [1, 2]
- **Revenue & Growth Drivers:** Anthropic's explosive growth is heavily fueled by enterprise adoption and its software-coding assistant (such as *Claude Code* and *Claude Opus 4.8*), leading to an annualized revenue run rate crossing **$47 billion**. OpenAI's annualized revenue is estimated between **$30 billion and $50+ billion**, driven heavily by mass consumer scale via ChatGPT. [1, 2, 3, 4]
- **Profitability & Burn Rate:** Both labs are burning billions on massive computing infrastructure. However, Anthropic has positioned itself closer to near-term enterprise

operating efficiencies and faster commercial monetization per contract, whereas OpenAI projects hundreds of billions in negative free cash flow through 2030. [1, 2, 3]

Claude Code versus OpenAI Codex

OpenAI relaunched Codex as a cloud-based coding agent in **May 2025** and reached a major parity and adoption inflection point when it rolled out major upgrades on April 30, 2026 (version `v0.128.0` with persisted `/goal` workflows and expanded free-tier access), leading to Codex surpassing Claude Code in weekly downloads by early **May 2026**. [1, 2]

Key Milestones Against Claude Code

- **May 2025:** OpenAI relaunched Codex as an autonomous cloud-based coding agent. [1]
- **April 30, 2026 (`v0.128.0`):** OpenAI released a major update featuring persisted `/goal` workflows, session persistence, and improved command-line and app-server integration. [1]
- **May 3, 2026:** Codex surpassed Claude Code in weekly downloads (reaching 86.1M vs. Claude Code's 7.2M) following aggressive free-tier access and rate-limit adjustments. [1]
- **October 2026 (`v0.162.0`):** OpenAI updated Codex CLI with managed Git worktree tools and task pinning powered by the **GPT-6.1 Sol** model (released September 29, 2026) to maintain feature parity with Anthropic's Claude Code updates. [1]

Chinese AI Milestones

Chinese AI models achieved several major global milestones in 2025 and 2026, shifting the competitive balance in large language models (LLMs) and open-source infrastructure: [1, 2, 3]

- **DeepSeek-R1 (January 2025):** DeepSeek launched an open-weight reasoning model matching OpenAI's o1 performance while costing less than $6 million to train, disrupting global AI cost assumptions. [1]
- **Global Developer Dominance (Mid-2026):** Chinese models from DeepSeek, Zhipu (Z.ai), and Alibaba accounted for 67% of tokens used on OpenRouter and 55% on Vercel, driven by low inference costs and strong coding performance. [1, 2]
- **Moonshot AI's Kimi K3 (July 2026):** Moonshot released Kimi K3 with 2.8 trillion parameters, making it one of the largest open-weight models announced and bringing open-source capabilities close to the 3-trillion-parameter mark. [1]
- **Alibaba's Qwen Ecosystem:** Alibaba's Qwen series exceeded 400 million downloads, solidifying Chinese leadership in open-source model distribution. [1, 2]
- **Massive Computing Infrastructure:** The China Mobile Hohhot Data Center in Inner Mongolia expanded to a capacity of 20,100 petaflops, with 88% of its power supplied by regional wind and solar green energy. [1]

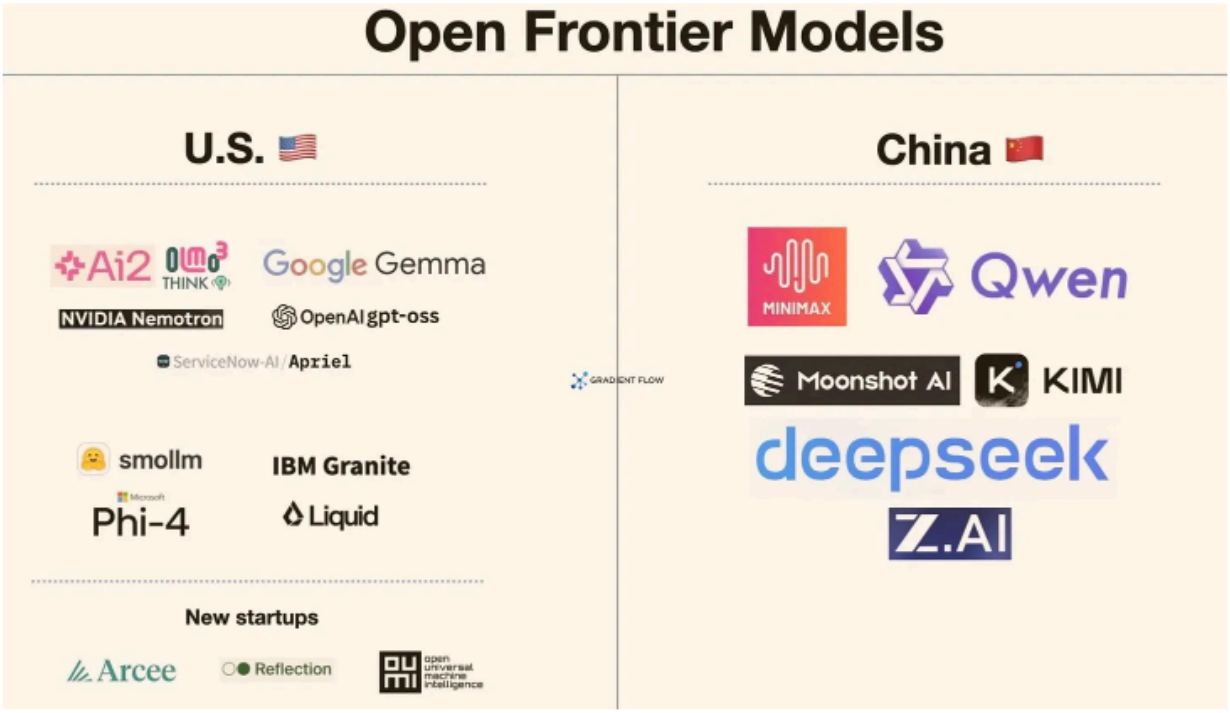


Image credit: Gradient Flow. (Ben Lorica 罗瑞卡)

https://www.ai-supremacy.com/p/milestones-of-china-in-ai-of-2025-deepseek-qwen

https://www.goldmansachs.com/insights/articles/competition-among-chinas-ai-models-is-intensifying-as-chip-sector-advances

AI Ancient History

1904 Marov Chains



Both Markov chains and modern Large Language Models (LLMs) share a foundational core: **next-token prediction**. They estimate a probability distribution to determine what word or token comes next in a sequence. However, they differ drastically in scale, memory, and how they represent language. [1, 2]

How Markov Chains Work

- **Definition:** A Markov chain is a mathematical system that transitions from one state to another based on fixed transition probabilities. [1, 2]
- **The Markov Property:** The future state depends *only* on the current state (or a short, fixed-size window of k previous tokens in an n-gram model). It has no long-term memory. [1, 2, 3]
- **Mechanism:** They rely on explicit, discrete frequency tables (lookup tables) built by counting how often word B follows word A in training text. [1, 2]
- **Limitations:** As you increase the memory window (e.g., 3-word or 4-word phrases), the number of possible combinations explodes, meaning most phrases become rare or completely unseen, leading to repetitive or incoherent text. [1, 2]

How Next-Token Prediction Works in LLMs

- **Definition:** Modern LLMs (like Transformers) also predict the next token, framing text generation as an autoregressive sequence task. [1, 2]
- **Continuous Latent Space:** Instead of rigid n-gram lookup tables, LLMs map words and tokens into high-dimensional continuous vector spaces (embeddings). [1]
- **Deep Context and Attention:** Using mechanisms like self-attention, an LLM looks across a massive context window (thousands or millions of tokens) rather than just the immediate prior word or fixed n-gram. [1, 2]
- **Generalization:** Because they learn semantic relationships rather than exact string matches, LLMs can generalize to novel phrasing, understand grammar, and maintain coherent, long-form logic. [1]

Key Differences at a Glance

Feature	Markov Chains (N-grams)	Modern LLMs (Transformers)
State Representation	Discrete string/word lookup tables	Continuous vector embeddings in latent space

Memory / Context	Very short and fixed (1 to 2 words typically)	Long context windows (thousands/millions of tokens)
Handling Unseen Data	Fails or repeats when an n-gram is missing	Generalizes smoothly via semantic interpolation
Output Quality	Often drifts into random or repetitive babble	Contextually relevant, coherent, and adaptive

See discussion on Hacker News for technical comparisons between traditional Markov chains and small language models. [1]

https://academy.claude.com/courses/ai-capabilities-and-limitations/try-it-out
https://www.taskade.com/wiki/foundations/markov-chain

1944: Neural Networks are theorized.
https://news.mit.edu/2017/explained-neural-networks-deep-learning-0414

1946 Monte Carlo Simulation

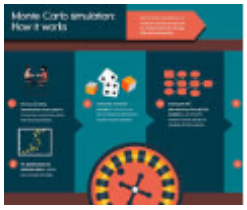


The Monte Carlo simulation was conceived in 1946 by mathematician Stanislaw Ulam and developed alongside John von Neumann during the Manhattan Project at Los Alamos National Laboratory. [1, 2, 3, 4, 5]

Key Milestones in Monte Carlo History

- **The Solitaire Spark (1946):** Stanislaw Ulam was recovering from brain surgery and playing the card game Solitaire. He wondered about the mathematical probability of winning a hand, and realized that running repeated trials to count wins was faster than solving it with complex deterministic formulas. [1, 2, 3]
- **Nuclear Physics & The Hydrogen Bomb:** Ulam shared his idea with John von Neumann. They used it to calculate neutron diffusion paths and fission behavior for nuclear weapons. [1, 2, 3]

- **The Name:** They named the technique "Monte Carlo" as a tribute to the famous casino in Monaco, because core reliance on chance and random sampling mirrored games like roulette. [1, 2, 3, 4]
- **Early Electronic Computing:** The first programs were run on early electronic computers like the ENIAC and the MANIAC at Los Alamos, marking a foundational moment for digital simulation. [1, 2, 3]
- **The Metropolis Algorithm (1952):** Nicholas Metropolis and colleagues expanded the method by introducing the Metropolis Monte Carlo algorithm for the first computer simulation of liquids. [1]
- **Modern Expansion:** Today, the technique powers risk assessment, financial portfolio modeling, supply chain forecasting, and modern artificial intelligence systems. [1, 2, 3]



AI enhances Monte Carlo simulations by automating model building, suggesting statistical probability distributions, generating synthetic training data, and interpreting results in plain language. [1, 2, 3]

How AI Integrates with Monte Carlo Simulations

- **Scaffolded Model Building:** Large Language Models (LLMs) and tools like Lumivero's @RISK Agent let analysts describe assets and business contexts in plain language to generate model structures and pick variables faster. [1]
- **Automated Python Scripting:** AI platforms like ChatGPT or DeepSeek can write complete Python scripts using `NumPy`, `Pandas`, and `Matplotlib` to run thousands of random sampling iterations and chart financial or stock valuations. [1]
- **Generative AI & Synthetic Data:** Generative AI models can train on real-world sequential or multi-level data to create realistic synthetic datasets for simulation studies, improving risk and predictive performance evaluations. [1, 2]
- **Result Interrogation:** Instead of manually digging through massive output tables, users can use natural language interfaces to query simulation results, ask what drives variance, or stress-test downside scenarios. [1]

Key Use Cases

- **Financial & Portfolio Forecasting:** Modeling asset returns, exchange rate risks, and retirement portfolios across thousands of random economic conditions.

- **Project Cost & Schedule Risk:** Projecting labor costs, material price volatility, and task duration ranges to prevent budget overruns.
- **Machine Learning & AI Evaluation:** Using Monte Carlo simulations in ML pipelines (such as the R.I.G.S. framework) to test agentic systems, prevent reward hacking, and evaluate model performance under diverse random inputs. [1, 2, 3, 4]

Here is a video demonstrating how to generate and execute a Monte Carlo simulation script using AI tools:

1948: Claude Shannon and information Theory

Claude Shannon's 1948 work, *"A Mathematical Theory of Communication,"* forms the mathematical foundation of modern Artificial Intelligence—particularly Large Language Models (LLMs) and deep learning. [1, 2, 3, 4, 5, 6]

---

1. Prediction and Data Compression are Mathematical Equivalents

- **The Core Insight:** Shannon demonstrated that the efficiency of compressing data is directly tied to how well you can predict it. If a sequence of text or data is highly predictable, it has low redundancy and can be compressed into a smaller number of bits. [1, 2, 3]
- **Connection to Modern AI:** Training an LLM to predict the next token (next-word prediction) is mathematically identical to building an optimal text compressor. The better the model predicts what comes next, the better it compresses the information. [1, 2, 3]

2. Cross-Entropy Loss Drives Model Training

- **The Core Insight:** Shannon introduced *entropy* to measure uncertainty or the average amount of information in a message (quantified via $-\log_2(probability)$). [1, 2, 3, 4]
- **Connection to Modern AI: Cross-entropy loss**—the primary loss function used to train deep neural networks and classification models—measures the difference between the model's predicted probability distribution and the true data distribution. Minimizing cross-entropy loss directly reduces uncertainty, mirroring Shannon's principles of optimal coding. [1, 2, 3]

3. Language Modeling and Markov Processes

- **The Core Insight:** Shannon applied **Markov processes** (where the probability of the next symbol depends on a finite history of preceding symbols) to model English text sequences. [1]
- **Connection to Modern AI:** Modern transformer architectures and autoregressive language models operate on this exact probabilistic paradigm. They treat language and token sequences as high-dimensional statistical distributions, scaling up Shannon's early n-gram and Markov experiments using massive neural network "lookup tables." [1]

4. Bracketing Meaning for Statistical Processing

- **The Core Insight:** Shannon famously stripped "meaning" or semantics out of his engineering equations, arguing that communication is purely a statistical and probabilistic selection from a set of possibilities. [1, 2, 3]
- **Connection to Modern AI:** Generative AI processes language, images, and audio through high-dimensional vector math and self-attention without possessing human consciousness or "felt understanding". They succeed precisely because they execute Shannon's pragmatic, statistical view of information. [1, 2]

5. Advanced Machine Learning Techniques

- **Kullback-Leibler (KL) Divergence:** Used heavily in Variational Autoencoders (VAEs), reinforcement learning from human feedback (RLHF), and diffusion models to measure how one probability distribution diverges from a second, baseline distribution. [1, 2]
- **Mutual Information:** Maximized in frameworks like InfoGAN to help neural networks learn disentangled, meaningful representations of data without direct supervision. [1]

https://machinelearningmastery.com/from-shannon-to-modern-ai-a-complete-information-theory-guide-for-machine-learning/

1970's Geof Hinton

During the **1970s**, **Geoffrey Hinton**—who later became widely celebrated as the **"Godfather of AI"** and co-won the 2024 **Nobel Prize in Physics**—began his foundational, highly contrarian work on artificial neural networks. [1, 2]

At a time when the broader field was stuck in a period of stagnation known as the "AI winter," Hinton stood practically alone in his belief that computers should mimic the biological structure of the human brain. [1, 2]

Academic Foundations & Psychology Roots

- **Psychology Degree (1970):** Hinton graduated from **King's College, Cambridge** in **1970** with a Bachelor of Arts in **experimental psychology**. This background fueled his

obsession with understanding how the physical human brain actually learns and stores memory, laying the groundwork for his biological approach to computer science.
- **Carpentry Interlude:** After graduating, he took a brief detour from academia and spent a year working as an apprentice **carpenter**. [1, 2, 3, 4]

Doctoral Research at Edinburgh (1972–1978)

- **The Standalone Program:** In **1972**, Hinton enrolled at the **University of Edinburgh**, which housed the United Kingdom's only postgraduate program for artificial intelligence at the time. [1]
- **Working in Obscurity:** The dominant paradigm of 1970s AI was "**Symbolic AI**" (logic-based, if-then rules programmed by humans). Neural networks were profoundly unpopular, written off by most researchers as an ineffective dead end. [1]
- **Battling His Advisor:** His PhD supervisor, Christopher Longuet-Higgins, was a proponent of symbolic AI and actively discouraged Hinton from studying neural networks, warning him that it would ruin his career. Hinton stubbornly pushed back, famously requesting "another six months" on a weekly basis to prove his brain-based simulations could work. [1, 2, 3, 4]
- **Graduation:** Despite the relentless skepticism, Hinton successfully defended his thesis on how the brain implements learning algorithms and was awarded his **PhD in Artificial Intelligence in 1978**. [1, 2]

Moving West (Late 1970s)

- **Postdoctoral Research (1978–1980):** Realizing that funding for his unconventional approach was dry in the UK, Hinton moved to the United States. By the late 1970s, he joined the **University of California, San Diego (UCSD)** as a visiting scholar in the Department of Psychology. [1, 2]
- **The PDP Group:** At UCSD, he aligned with the **Parallel Distributed Processing (PDP) group**—a collection of like-minded cognitive scientists, psychologists, and linguists. This collaborative environment directly set the stage for his massive breakthroughs in the 1980s, including the famous co-development of the **backpropagation algorithm** in 1986. [1, 2, 3]